Chapter 3: ARM Cortex-M4 programming

- Introduction
- ARM Cortex-M
- STM32F429 Microcontroller
- STM32F429I-DISCOVERY Development Kit
- Programming with STM32F429

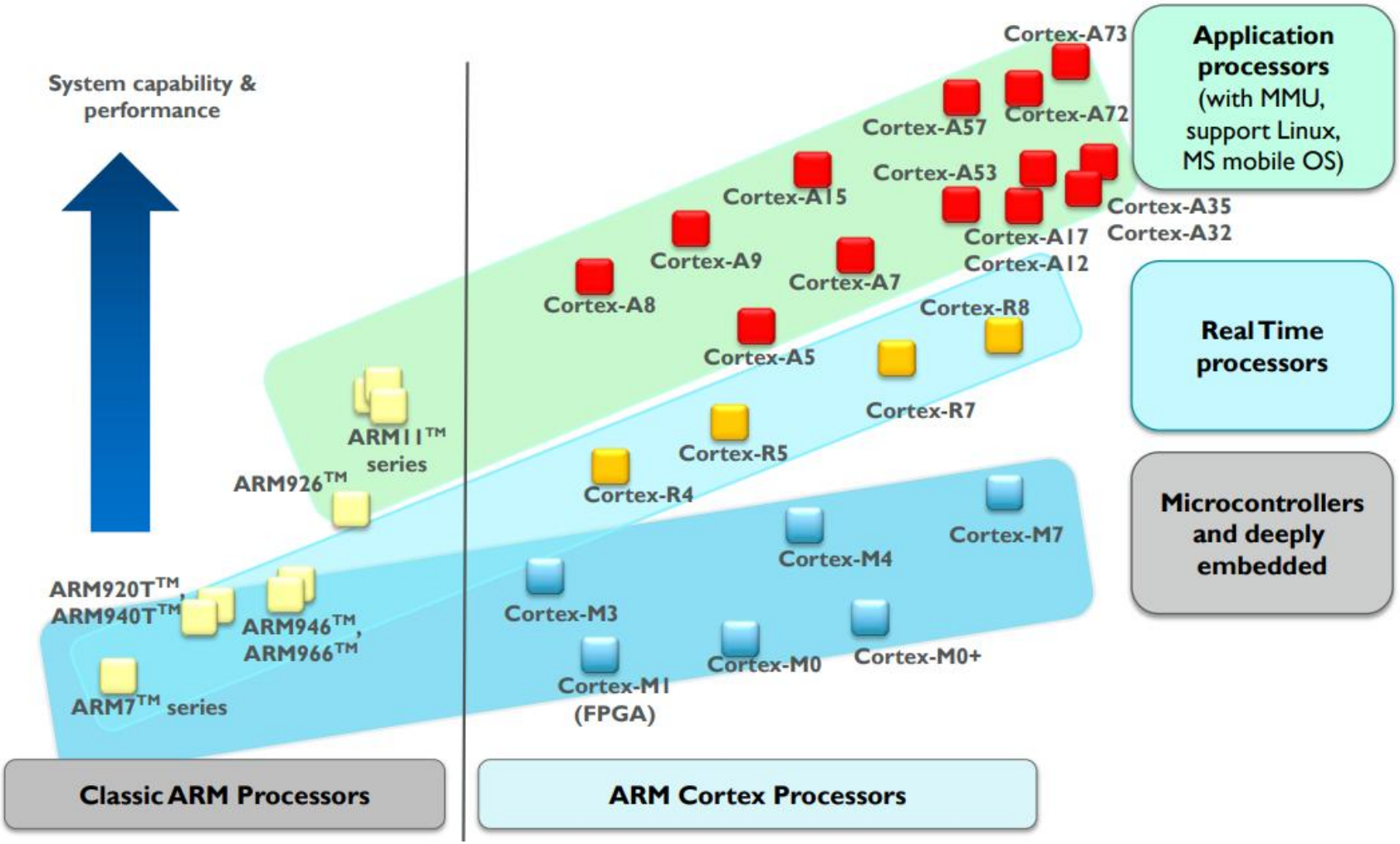
We're going to read a lot of datasheet!

8 bit vs 32 bit microcontrollers

- 8 bit MCUs: (8051, PIC, AVR, STM8…)
  - Low cost, simple hardware and software.
  - Low performance (10-20 MHz), small data bus (8 bit), low availability of peripherals.
  - Suitable for simple and low-cost applications: toys, consumer products,...
  - VD: Arduino classic boards
- 32 bit MCUs: (ARM-based, PIC32, AVR32,...)
  - 32 bit instruction set and operands, 32 bit data bus: much higher performance.
  - High clock rate (100-200 MHz).
  - Large availability of peripherals.
  - Suitable for more complicated system with multi-task, data acquisition and processing, and better connectivity.
- Most popular 32-bit MCU: ARM-based microcontroller

ARM-powered products

ARM processor portfolio



Classification of ARM processors

ARM Cortex Processors

- ARM Cortex-**A** family:

  Applications processors for full-fledge high performance (Linux, Android…) with clock rate > 1 GHz.

- ARM Cortex-**R** family:

  Embedded processors for real-time systems that requires absolute stability and safety in operation, clock rate usually of 200 MHz – 1 GHz.

- ARM Cortex-**M** family:

  Microcontroller for embedded system, clock rate < 200 MHz.

Cortex family

Cortex-A8

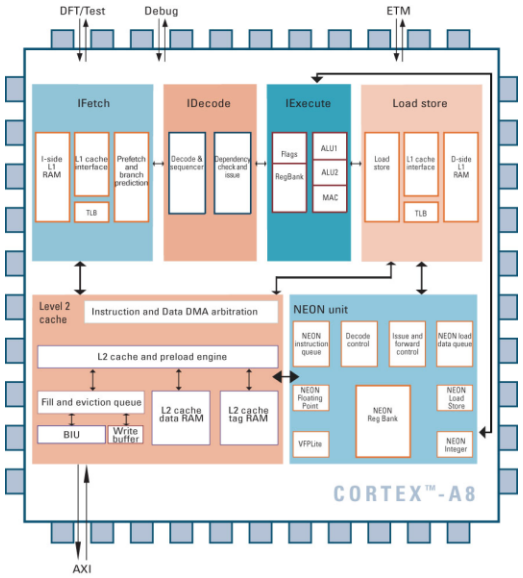
- Architecture v7A
- MMU
- AXI
- VFP & NEON support

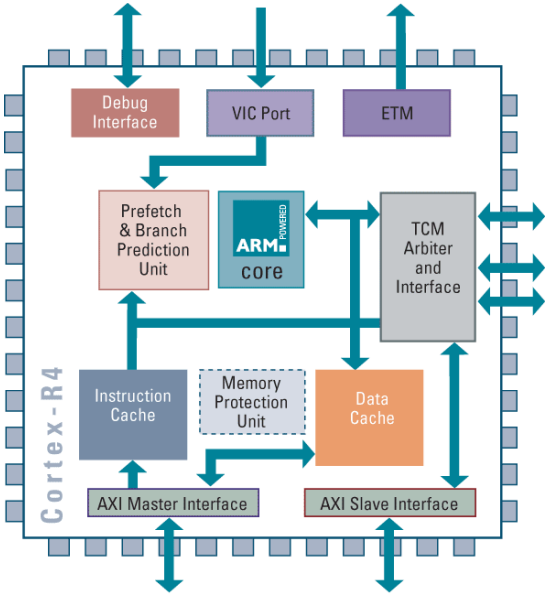
Cortex-R4

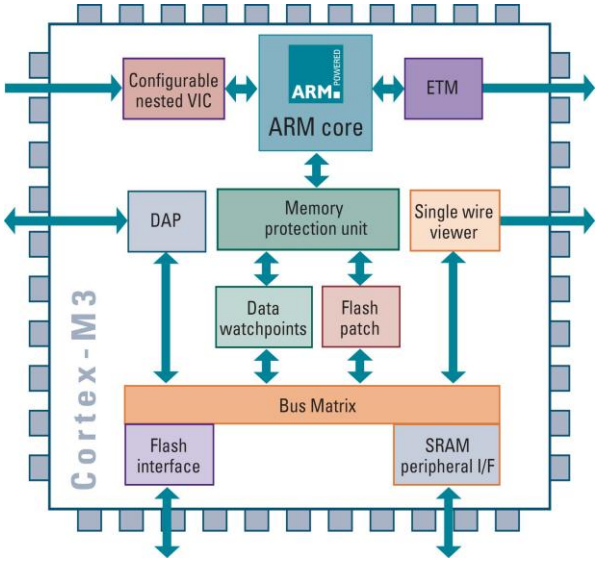
- Architecture v7R
- MPU (optional)
- AXI
- Dual Issue

Cortex-M3

- Architecture v7M
- MPU (optional)
- AHB Lite & APB

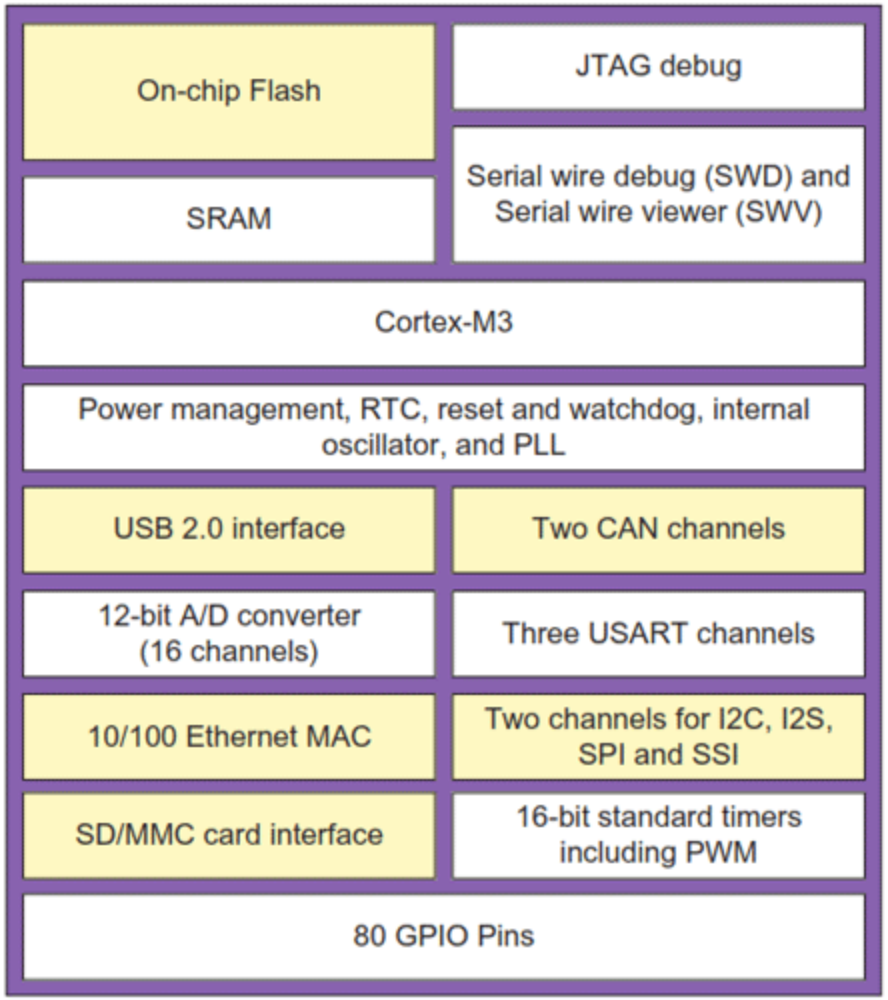






ARM Cortex-M

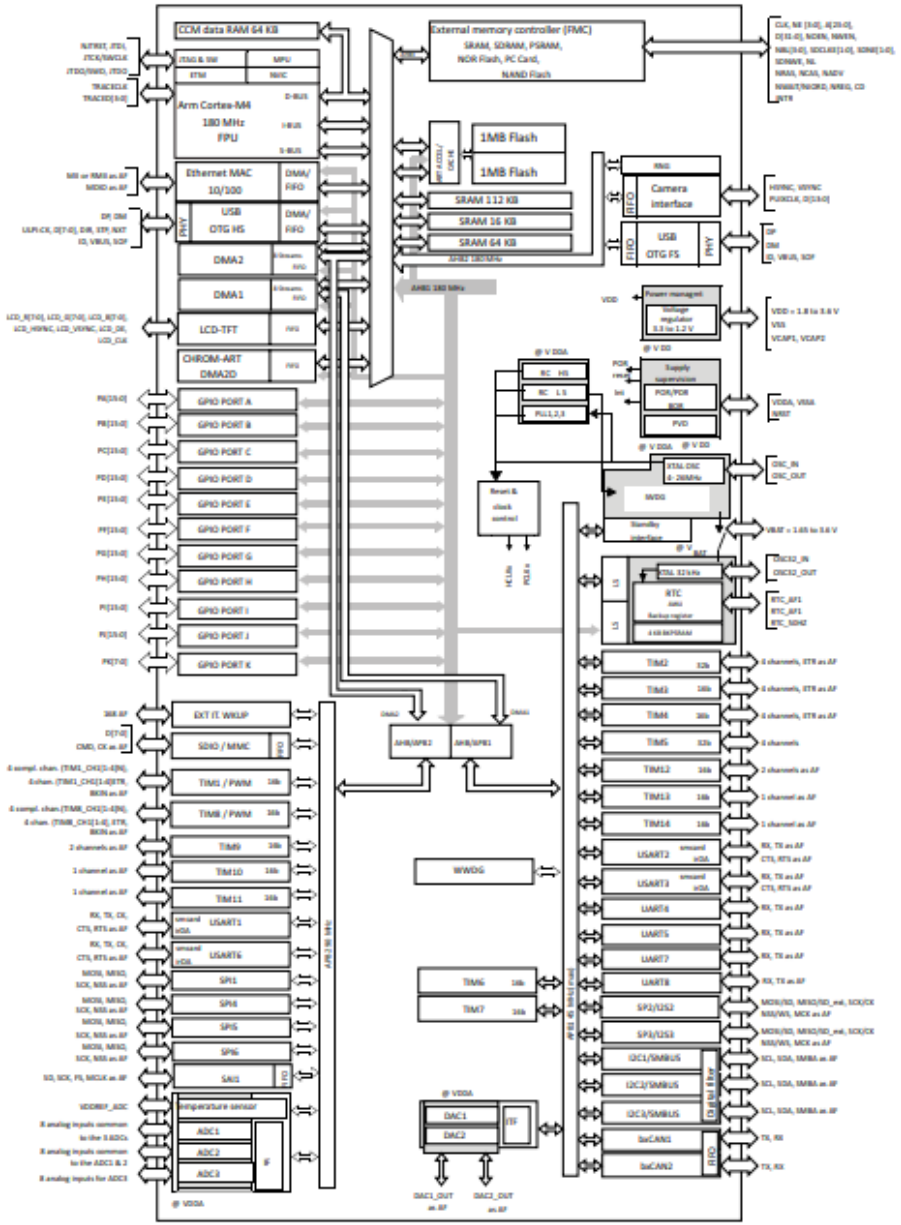
- Some typical components/peripherals on Cortex-M MCU



Some MCUs designed based on Cortex-M4

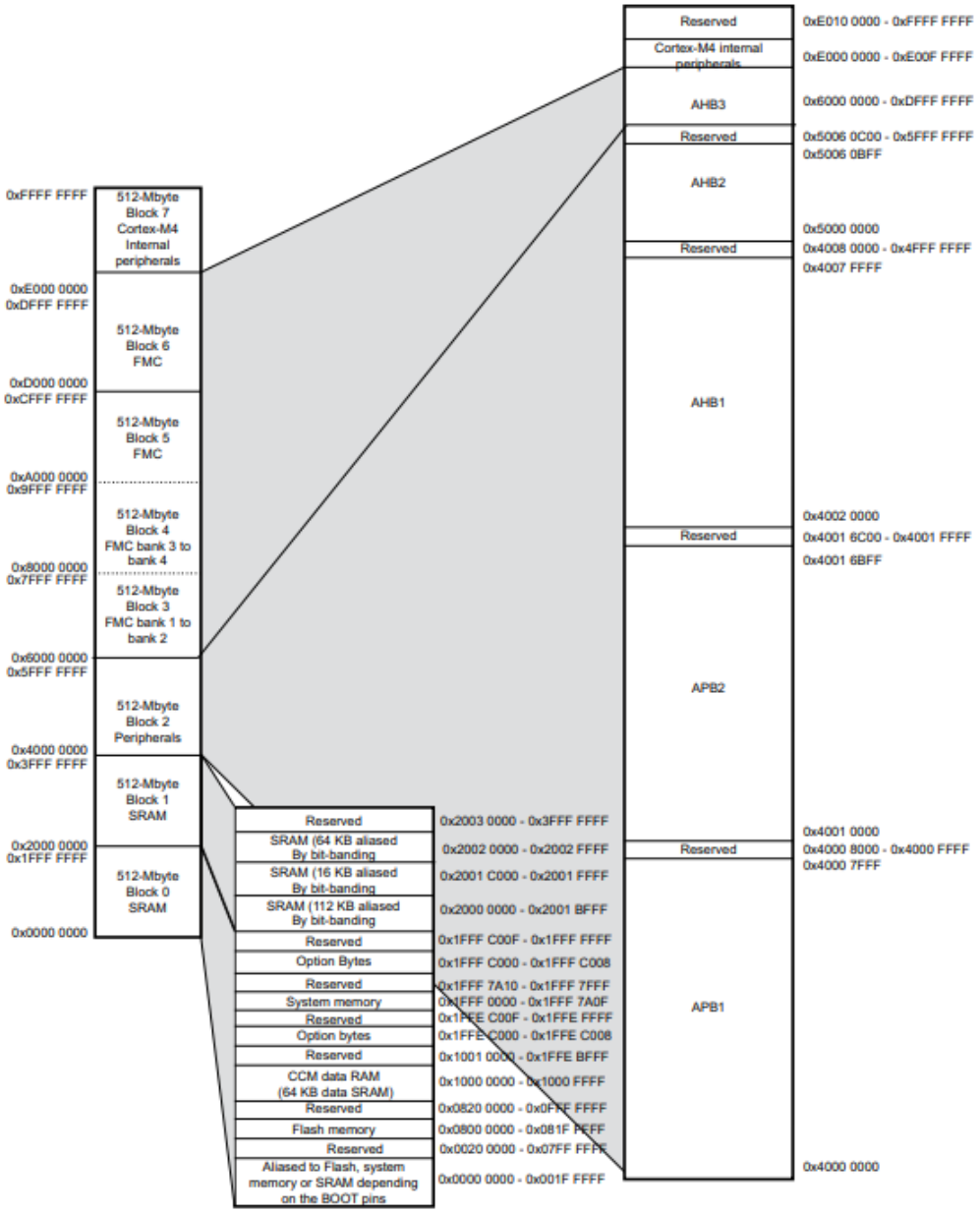
- Analog Devices ADUCM4050
- Cypress 6200, FM4
- Infineon XMC4000
- Maxim Darwin
- Microchip (Atmel) SAM4C/4E/G5, SAMD5/E5x
- Nordic nRF52
- Nuvoton NuMicro M480
- NXP LPC4000, LPC4300 LPC54000
- NXP (Freescale) Kinetis K, V3, V4
- Renesas S3, S5, S7, RA4, RA6
- Silicon Labs (Energy Micro) EFM32 Wonder
- ST STM32 F3, F4, L4, L4+, G4, WB
- Texas Instruments LM4F, TM4C, MSP432, CC13x2R, CC1352P, CC26x2R
- Toshiba TX04

STM32F429xxx block diagram



STM32F429xx

Memory map



Important notices

- High CPU clock rate: requires a software architecture to efficiently use the CPU's capabilities
  - Polling in main loop
  - Interrupt handling
  - Interrupt-based and DMA for I/O
  - Task management with embedded OS

    →We'll learn these things gradually
- A lot of peripherals, each requires very complicated configuration: requires special design tools

  →STM32CubeIDE
- A lot of CPUs from different vendors, but share the same Cortex-M cores: requires libs/middleware for scalability and reusing code.

  → CMSIS, HAL, LL libraries...

STM32F429ZI MCU

- Docs:
  - STM32F427xx STM32F429xx datasheet
  - STM32F Reference
- STM32F429ZIT6 is a product in STM32F429 series
  - 180 MHz max CPU (4-26 MHz crystal)
  - 2 MB Flash, 256 KB + 4 KB SRAM
  - FPU + DSP core
  - Built-in Ethernet, USB
  - 17 Timers (16/32 bits, 180 MHz)
  - 3x12 bits ADCs (3x2.4 MSPS, 24 channels)
  - 21 communication interfaces (SPI, I2C, I2S…)
  - Camera interface, LCD interface, DRAM interface…
  - RTC, CRC, Random generator…
  - …

STM32F429ZI MCU: GPIO pins

- LQFP144 package: 144 pins
- GPIO ports: PA, PB, PC, PD, PE, PF, PG, PH
- Each port has multiple pins, each pin can perform multiple functions

Figure 13. STM32F42x LQFP144 pinout

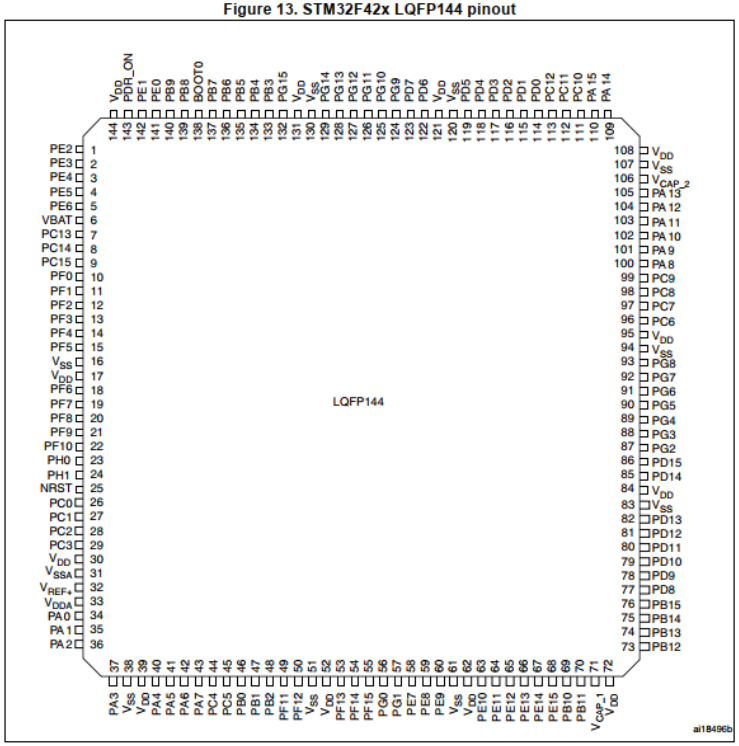


Table 10. STM32F427xx and STM32F429xx pin and ball definitions

Pin number								Pin name (function after reset)(1)	Pin type	I / O structure	Notes	Alternate functions	Additional functions
LQFP100	LQFP144	UFBGA169	UFBGA176	LQFP176	WLCSP143	LQFP208	TFBGA216						
1	1	B2	A2	1	D8	1	A3	PE2	I/O	FT	-	TRACECLK, SPI4_SCK, SAI1_MCLK_A, ETH_MII_TXD3, FMC_A23, EVENTOUT	-
2	2	C1	A1	2	C10	2	A2	PE3	I/O	FT	-	TRACED0, SAI1_SD_B, FMC_A19, EVENTOUT	-
3	3	C2	B1	3	B11	3	A1	PE4	I/O	FT	-	TRACED1, SPI4_NSS, SAI1_FS_A, FMC_A20, DCMI_D4, LCD_B0, EVENTOUT	-

See Datasheet for other pins

STM32F429ZI: GPIP pins alternate functions

Table 12. STM32F427xx and STM32F429xx alternate function mapping

Port		AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7	AF8	AF9	AF10	AF11	AF12	AF13	AF14	AF15
		SYS	TIM1/2	TIM3/4/5	TIM8/9/10/11	I2C1/2/3	SPI1/2/3/4/5/6	SPI2/3/SAI1	SPI3/USART1/2/3	USART6/UART4/5/7/8	CAN1/2/TIM12/13/14/LCD	OTG2_HS/OTG1_FS	ETH	FMC/SDIO/OTG2_FS	DCMI	LCD	SYS
Port A	PA0	-	TIM2_CH1/TIM2_ETR	TIM5_CH1	TIM8_ETR	-	-	-	USART2_CTS	UART4_TX	-	-	ETH_MII_CRS	-	-	-	EVENTOUT
	PA1	-	TIM2_CH2	TIM5_CH2	-	-	-	-	USART2_RTS	UART4_RX	-	-	ETH_MII_RX_CLK/ETH_RMII_REF_CLK	-	-	-	EVENTOUT
	PA2	-	TIM2_CH3	TIM5_CH3	TIM9_CH1	-	-	-	USART2_TX	-	-	-	ETH_MDIO	-	-	-	EVENTOUT
	PA3	-	TIM2_CH4	TIM5_CH4	TIM9_CH2	-	-	-	USART2_RX	-	-	OTG_HS_ULPI_D0	ETH_MII_COL	-	-	LCD_B5	EVENTOUT
	PA4	-	-	-	-	-	SPI1_NSS	SPI3_NSS/I2S3_WS	USART2_CK	-	-	-	-	OTG_HS_SOF	DCMI_HSYNC	LCD_VSYNC	EVENTOUT
	PA5	-	TIM2_CH1/TIM2_ETR	-	TIM8_CH1N	-	SPI1_SCK	-	-	-	-	OTG_HS_ULPI_CK	-	-	-	-	EVENTOUT
	PA6	-	TIM1_BKIN	TIM3_CH1	TIM8_BKIN	-	SPI1_MISO	-	-	-	TIM13_CH1	-	-	-	DCMI_PIXCLK	LCD_G2	EVENTOUT
	PA7	-	TIM1_CH1N	TIM3_CH2	TIM8_CH1N	-	SPI1_MOSI	-	-	-	TIM14_CH1	-	ETH_MII_RX_DV/ETH_RMII_CRS_DV	-	-	-	EVENTOUT
	PA8	MCO1	TIM1_CH1	-	-	I2C3_SCL	-	-	USART1_CK	-	-	OTG_FS_SOF	-	-	-	LCD_R6	EVENTOUT
	PA9	-	TIM1_CH2	-	-	I2C3_SMBA	-	-	USART1_TX	-	-	-	-	-	DCMI_D0	-	EVENTOUT
	PA10	-	TIM1_CH3	-	-	-	-	-	USART1_RX	-	-	OTG_FS_ID	-	-	DCMI_D1	-	EVENTOUT
	PA11	-	TIM1_CH4	-	-	-	-	-	USART1_CTS	-	CAN1_RX	OTG_FS_DM	-	-	-	LCD_R4	EVENTOUT
	PA12	-	TIM1_ETR	-	-	-	-	-	USART1_RTS	-	CAN1_TX	OTG_FS_DP	-	-	-	LCD_R5	EVENTOUT

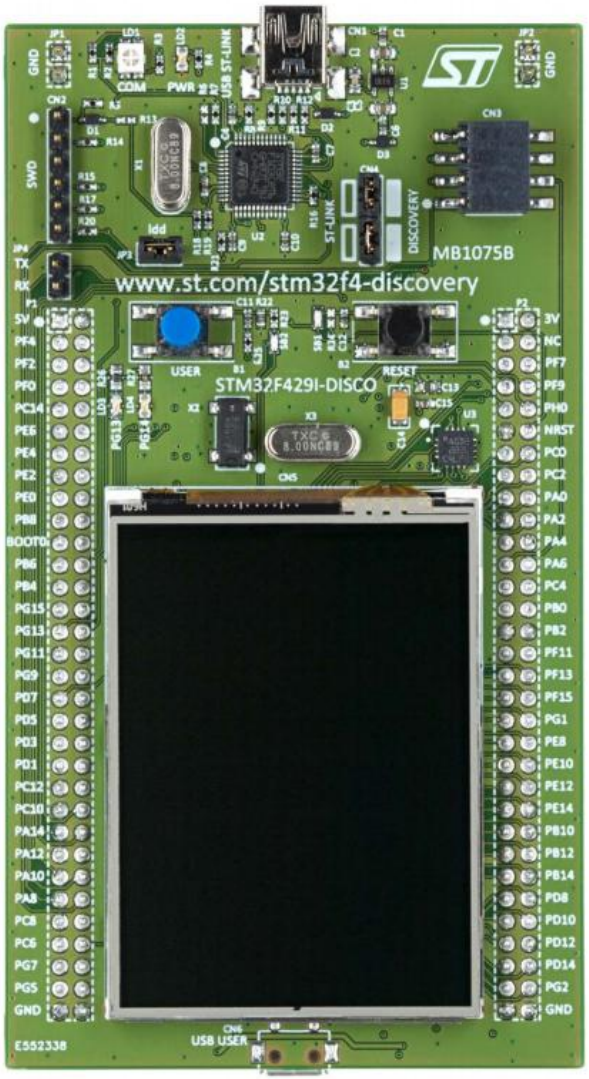
See Datasheet for other ports

Exercise: STM32F429ZIT6

- Get the below info from STM32F429ZIT6 datasheet
  - How many GPIO ports are available?
  - How many pins are available on each port?
  - What is the normal operating voltage of the MCU?

STM32F429I-DISCOVERY development kit

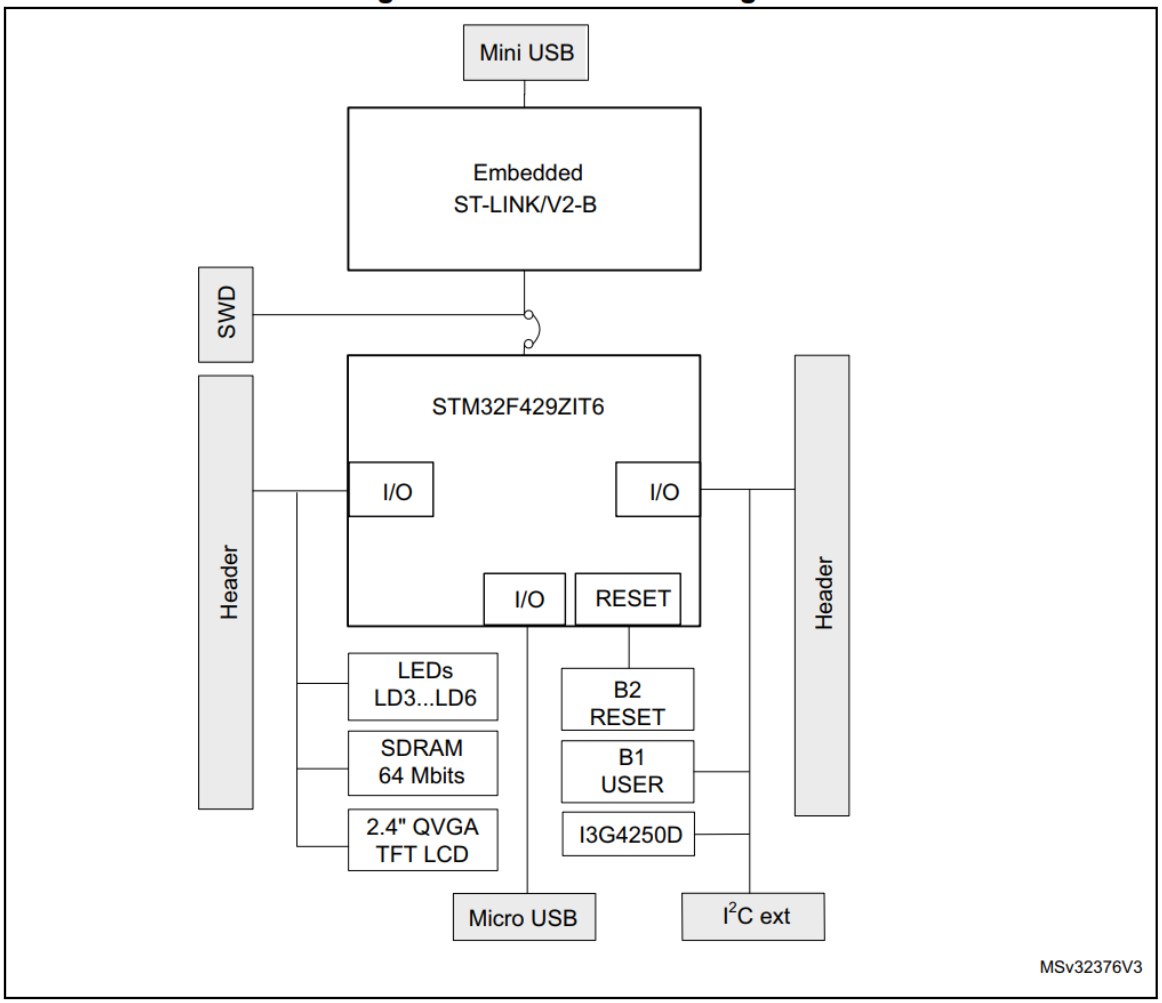
- STM32F429ZIT6 ARM Cortex-M4
- 2 MB flash, 256 KB SRAM
- 8MB SDRAM
- On-board ST-Link debugger
- 2.4” QVGA touch LCD
- 6 LEDs (2 user LEDs)
- 3 axis gyros
- …



STM32F429I-DISCOVERY development kit

- Hardware overview

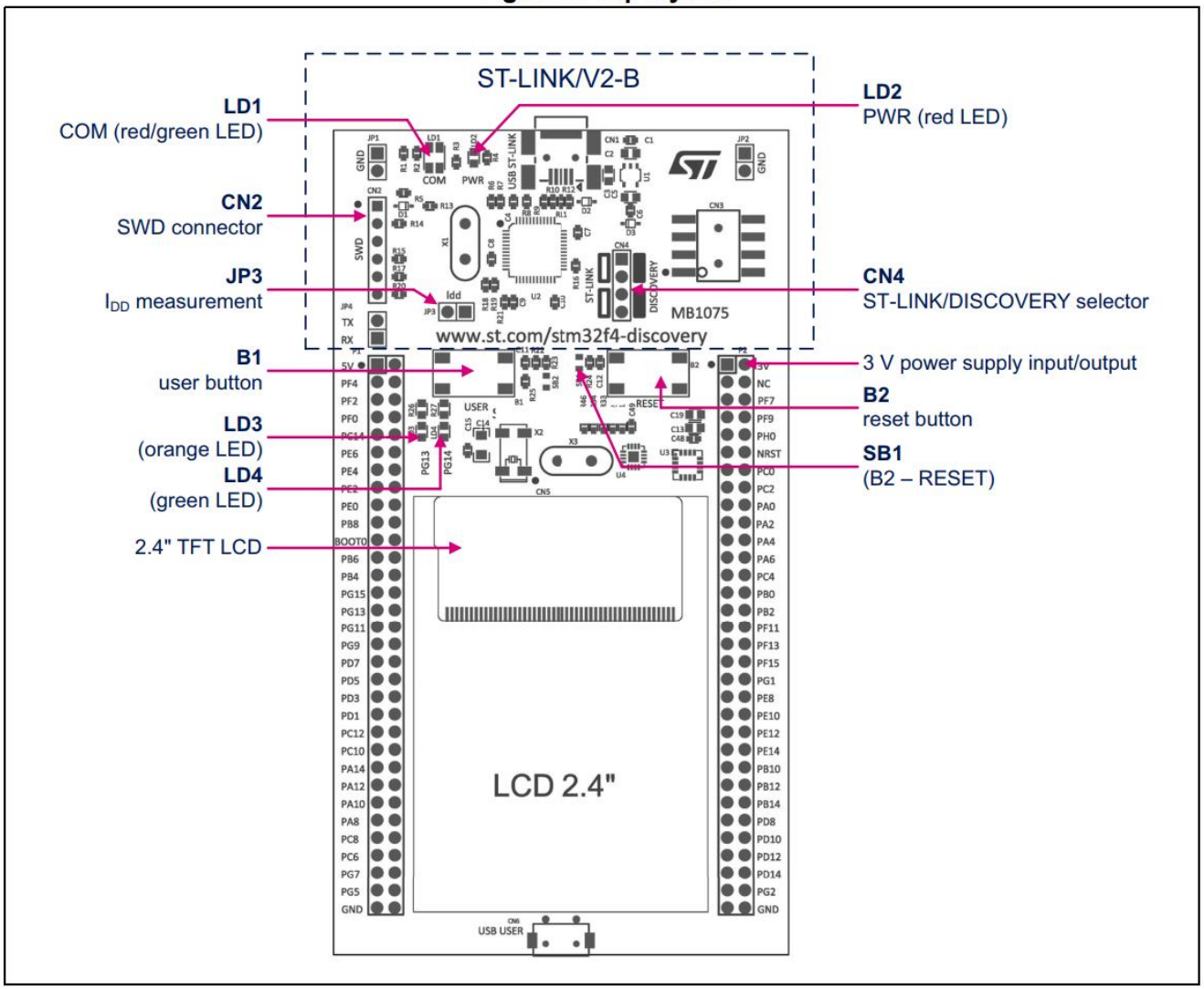
Figure 2. Hardware block diagram



STM32F429I-DISCOVERY development kit

- Hardware overview

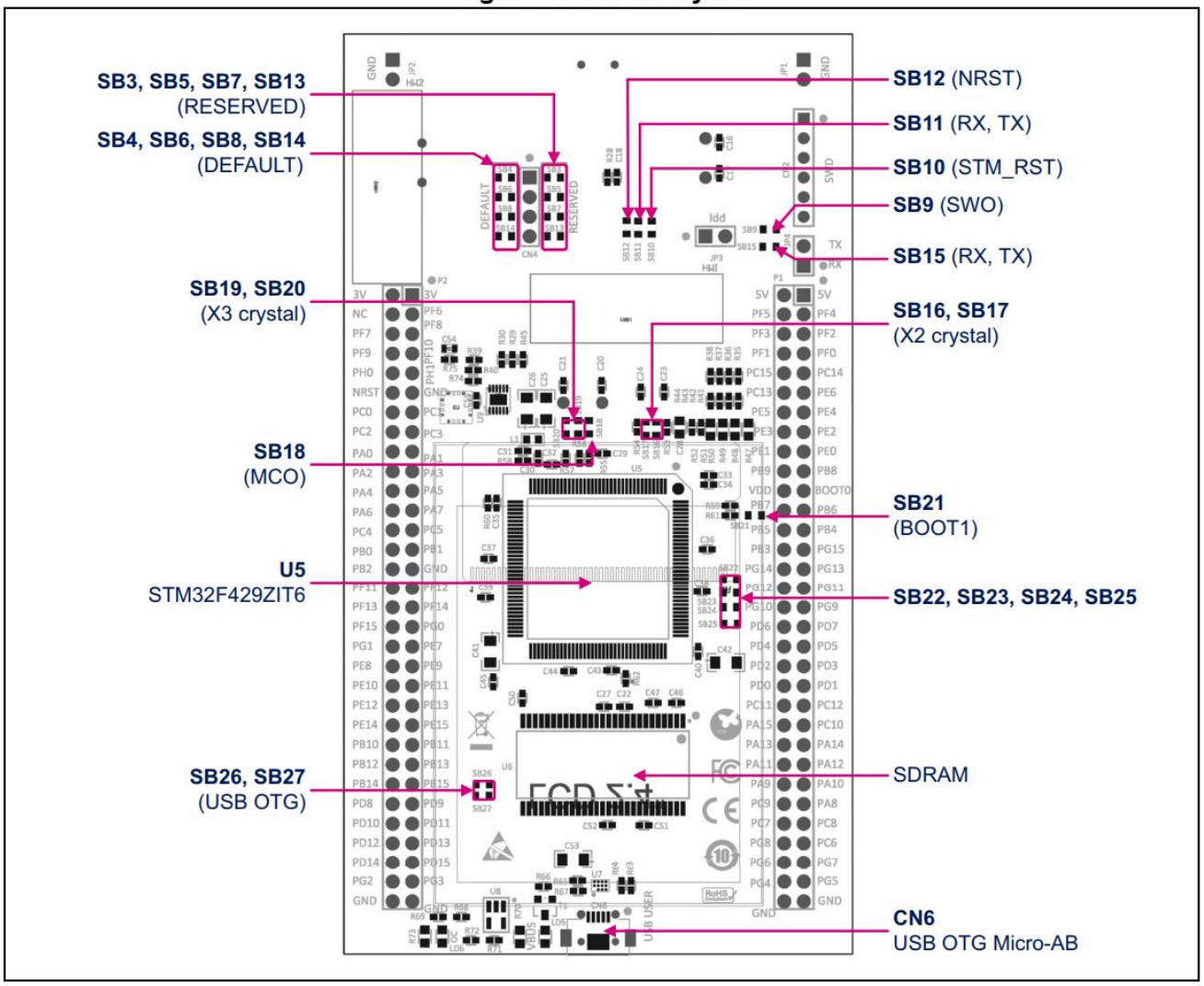
Figure 3. Top layout



STM32F429I-DISCOVERY development kit

- Hardware overview

Figure 4. Bottom layout



STM32F429I-DISCOVERY development kit

- Pin mapping

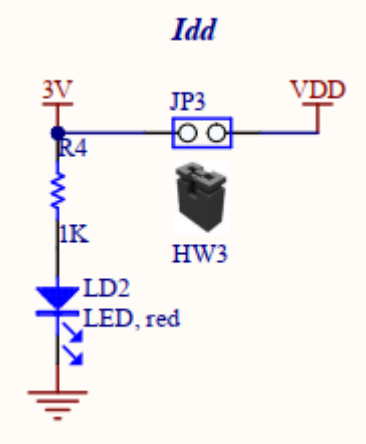
Table 7. STM32 pin description versus board functions

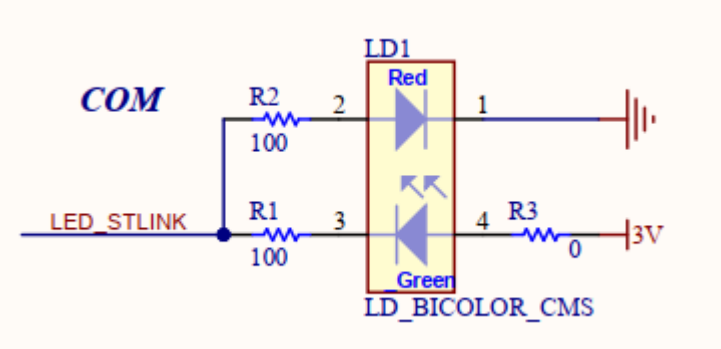
STM32 pin		Board functions																		
Main function	LQFP144	System	VCP	SDRAM	LCD-TFT	LCD-RGB	LCD-SPI	I3G4250D	USB	LED	Push-button	I^2C Ext	Touch panel	Free I/O	Power supply	CN2	CN3	CN6	P1	P2
BOOT0	138	BOOT0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	21	-
NRST	25	NRST	-	-	RESET	RESET	RESET	-	-	-	B2	-	-	-	-	5	-	-	-	12
PA0	34	-	-	-	-	-	-	-	-	-	B1	-	-	-	-	-	-	-	-	18
PA1	35	-	-	-	-	-	-	INT1	-	-	-	-	-	-	-	-	-	-	-	17
PA2	36	-	-	-	-	-	-	INT2	-	-	-	-	-	-	-	-	-	-	-	20
PA3	37	-	-	-	DB3	B5	-	-	-	-	-	-	-	-	-	-	-	-	-	19
PA4	40	-	-	-	VSYNC	VSYNC	-	-	-	-	-	-	-	-	-	-	-	-	-	22
PA5	41	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	21
PA6	42	-	-	-	DB6	G2	-	-	-	-	-	-	-	-	-	-	-	-	-	24
PA7	43	-	-	-	-	-	-	-	-	-	-	I2C_EXT_RST	-	-	-	-	4	-	-	23

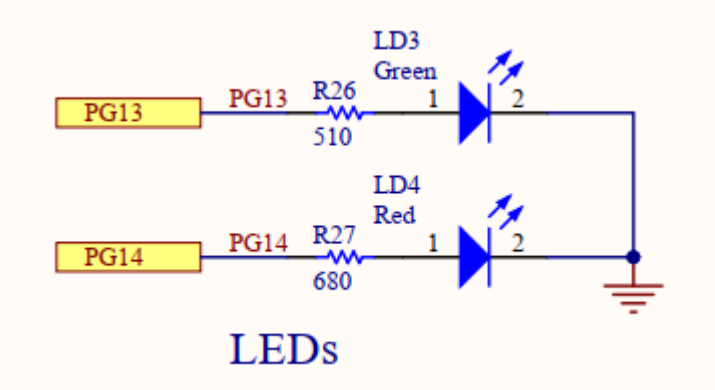
Exercise

- See the kit's User manual and Schematics to find the below info:
  - How are LEDs LD1 – LD4 controlled? When are they turned on/off?
  - Which pins on STM32F429 are connected to B1 và B2 buttons? What are the functions of these buttons? How do they work?
  - Which pins on the P1 and P2 extension headers are connected to the Touch Panel?
  - Which pins on the P1 and P2 extension headers are connected to the gyroscope I3G4250D?
  - Which pins on the P1 and P2 extension headers are not connected to any peripherals on the board?

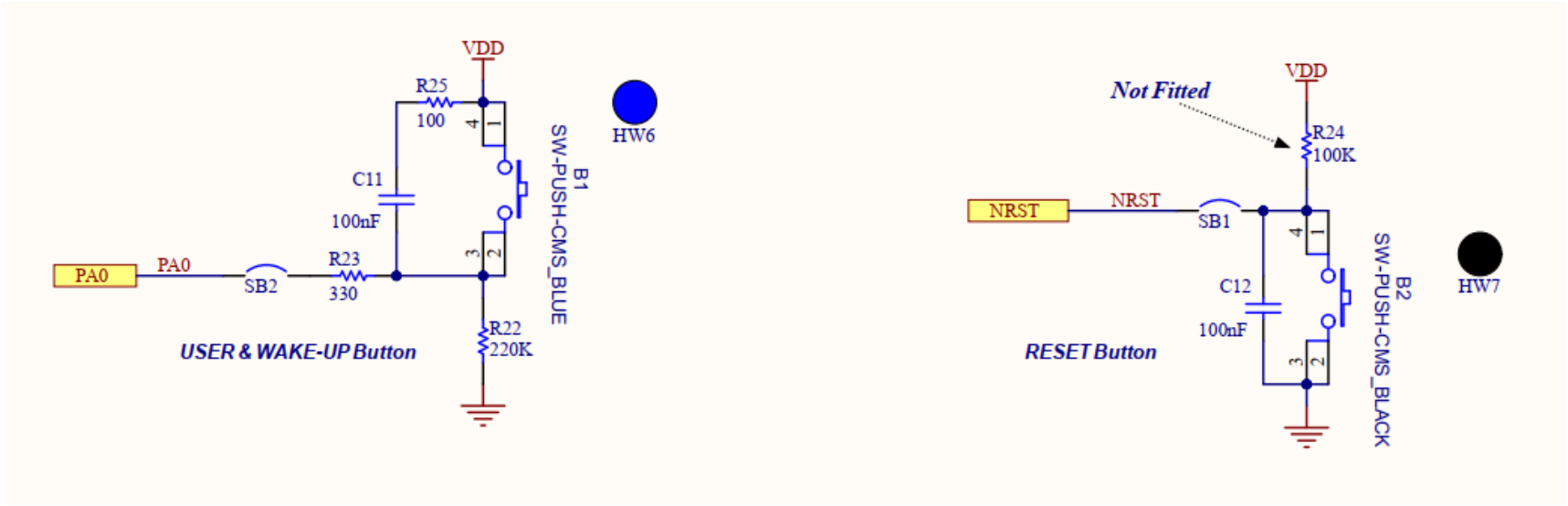
LD1 – LD4 LEDs


Idd
3V
JP3
VDD
R4
1K
HW3
LD2
LED, red


COM
LD1
Red
R2
100
2
1
LED_STLINK
R1
100
3
4
R3
0
3V
Green
LD_BICOLOR_CMS


LD3
Green
PG13
PG13
R26
510
1
2
LD4
Red
PG14
PG14
R27
680
1
2
LEDs

Push buttons



General Purpose Input/Output

- GPIO is the most basic peripherals of MCUs.
- GIOP pins (chân vào ra) are digital signal pins that supports input and output.
  - Input: receive signals from buttons, switches, sensors…
  - Output: send signals to drive LEDs, or to control other ICs/devices.
  - Logical values 0 and 1 correspond to low and high voltage signals.
- GPIO ports: groups of multiple pins (normally 8 to 16).
- GPIO pins can be multiplexed with other alternative functions (communications, analog signal,…).

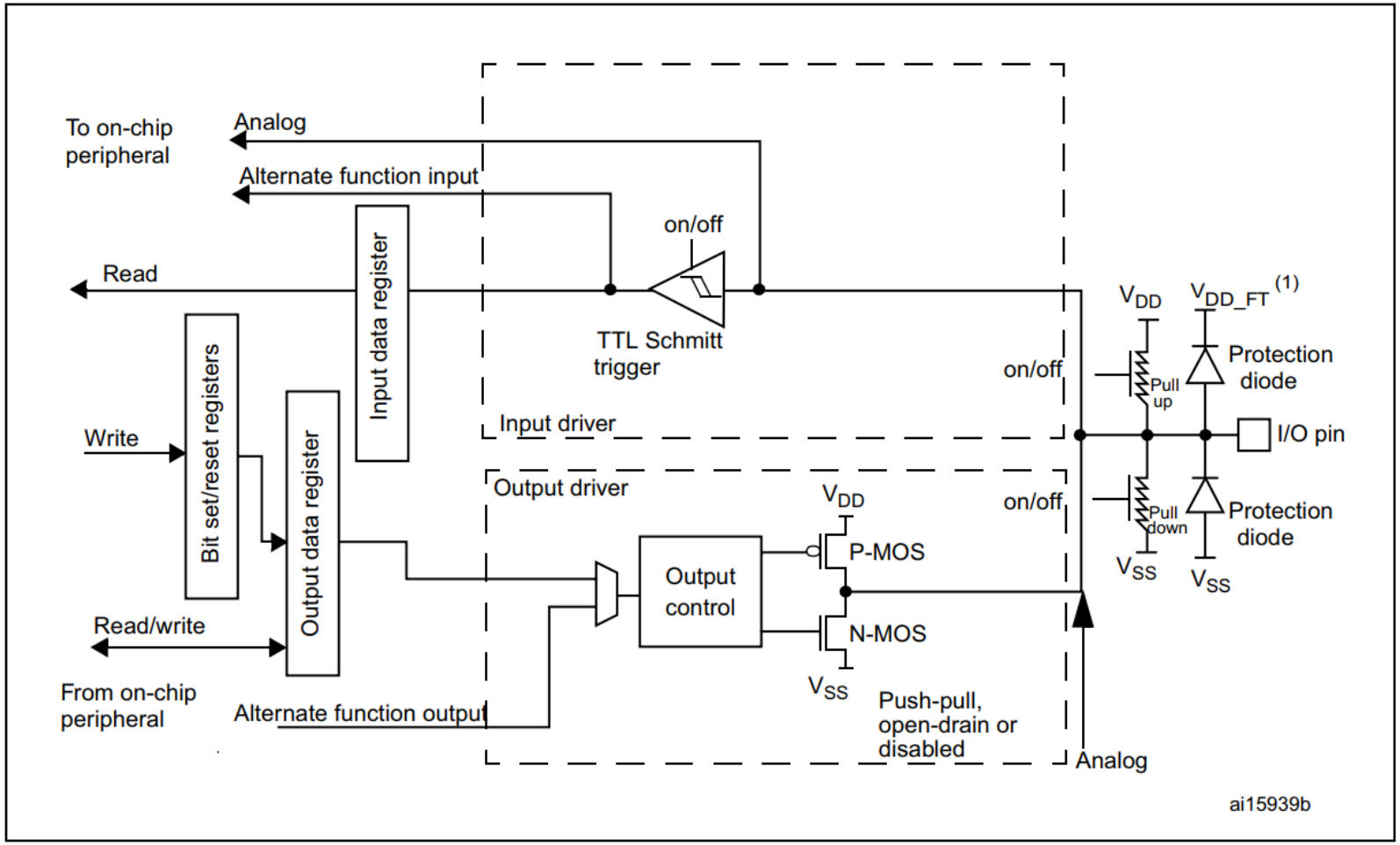
General Purpose Input/Output

- GPIO pin modes
  - Input floating
  - Input pull-up
  - Input-pull-down
  - Analog
  - Output open-drain with pull-up or pull-down capability
  - Output push-pull with pull-up or pull-down capability
  - Alternate function push-pull with pull-up or pull-down capability
  - Alternate function open-drain with pull-up or pull-down capability

GPIO pin structures

- Source: STM32F429 reference

Figure 25. Basic structure of a five-volt tolerant I/O port bit



GPIO pin structures

- Working modes

Figure 28. Input floating/pull up/pull down configurations

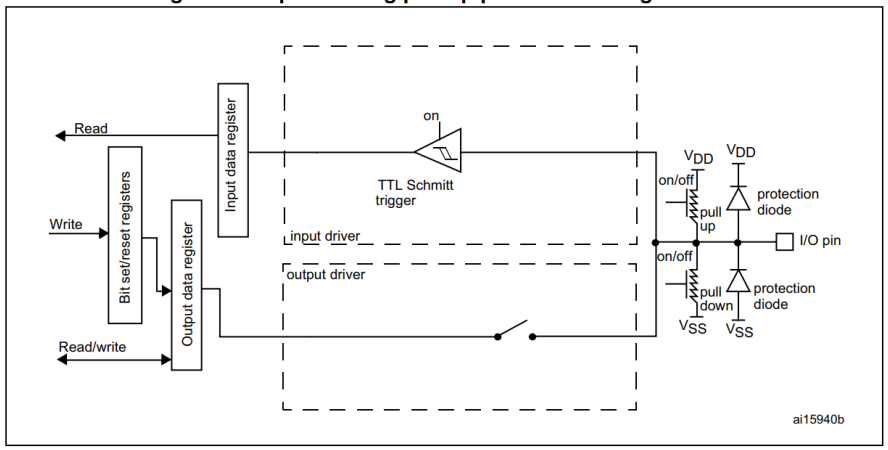


Figure 29. Output configuration

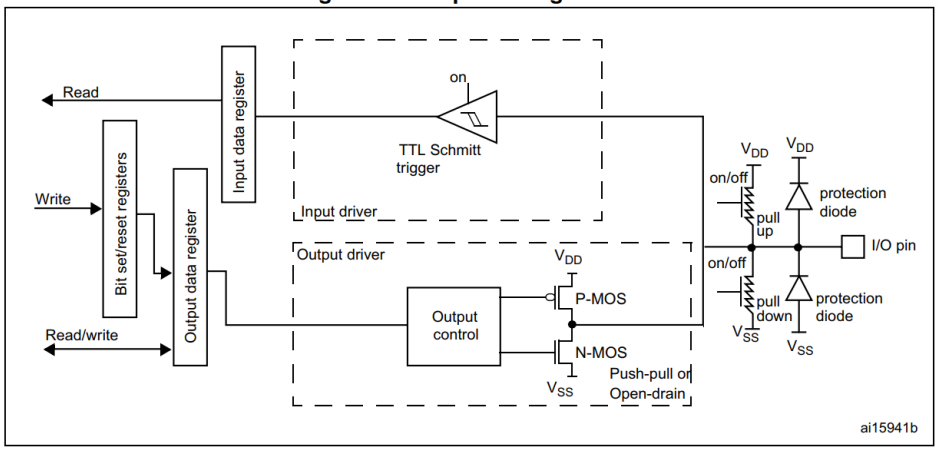


Figure 30. Alternate function configuration

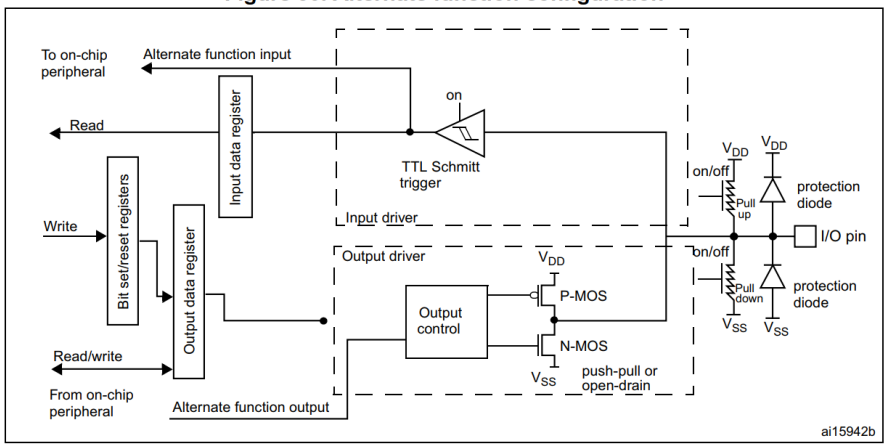
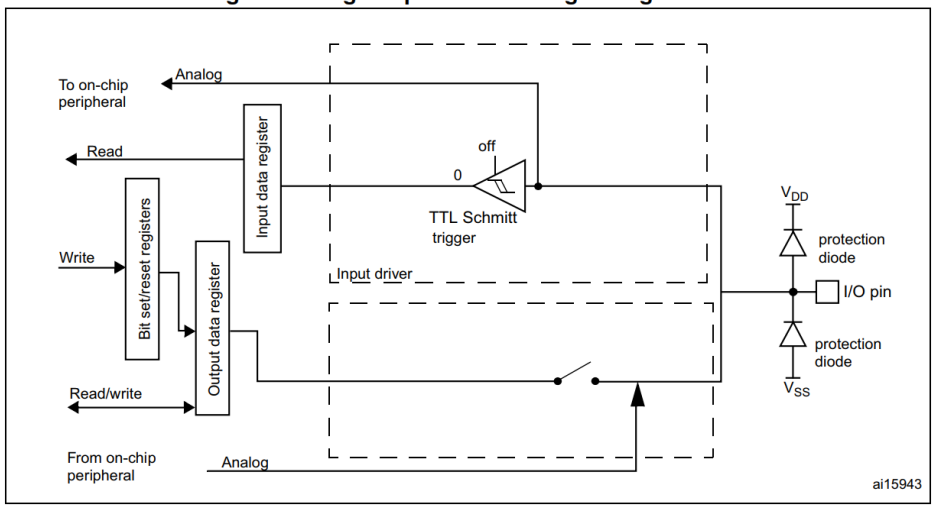


Figure 31. High impedance-analog configuration



*Nếu **đọc** vào từ một **output GPIO pin** thì sao?*
*Nếu **ghi** ra một **input GPIO pin** thì sao?*

Programming with GPIO

- Each GPIO port is controlled by a number of 32 bit registers
  - 4 configuration registers: set working mode for each GPIO pin
    - GPIOx_MODER,
    - GPIOx_OTYPER,
    - GPIOx_OSPEEDR,
    - GPIOx_PUPDR
  - 2 data registers: hold the read/write buffers of each pin
    - GPIOx_IDR,
    - GPIOx_ODR
  - Set/reset register: GPIOx_BSRR
  - Locking register: GPIOx_LCKR
  - 2 alternate function selection registers
    - GPIOx_AFRH,
    - GPIOx_AFRL

Example: Port bit configuration

- See Reference for more details about each register

Table 36. Port bit configuration table(1)

MODER(i) [1:0]	OTYPER(i)	OSPEEDR(i) [B:A]	PUPDR(i) [1:0]		I/O configuration	
01	0	SPEED [B:A]	0	0	GP output	PP
	0		0	1	GP output	PP + PU
	0		1	0	GP output	PP + PD
	0		1	1	Reserved	
	1		0	0	GP output	OD
	1		0	1	GP output	OD + PU
	1		1	0	GP output	OD + PD
	1		1	1	Reserved (GP output OD)	

Programming with GPIO

- GPIO ports can be controlled directly by read/write the port registers.
  - ➔Very very difficult task to comprehend all these configuration registers.
- GPIO drivers
  - HAL (Hardware Abstraction Layer): high-level and feature-oriented APIs with a high-portability level, hides the MCU and peripheral complexity from the end-user.
  - LL (Low-level) drivers offers low-level APIs at register level, with better optimization but less portability, require deep knowledge of the MCU and peripheral specifications.

Programming with GPIO

- HAL_GPIO functions

Initialization and de-initialization functions

This section provides functions allowing to initialize and de-initialize the GPIOs to be ready for use.

This section contains the following APIs:

- *HAL_GPIO_Init()*
- *HAL_GPIO_DeInit()*

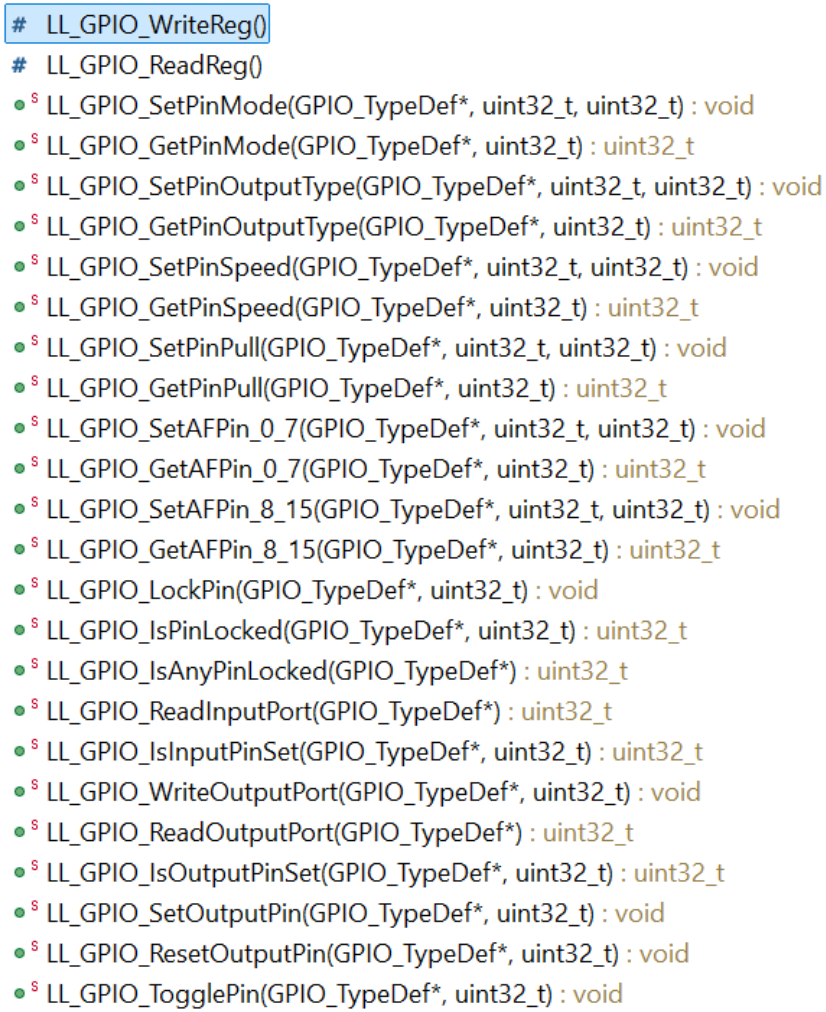
IO operation functions

This section contains the following APIs:

- *HAL_GPIO_ReadPin()*
- *HAL_GPIO_WritePin()*
- *HAL_GPIO_TogglePin()*
- *HAL_GPIO_LockPin()*
- *HAL_GPIO_EXTI_IRQHandler()*
- *HAL_GPIO_EXTI_Callback()*

Programming with GPIO

- LL_GPIO functions



Tool chain

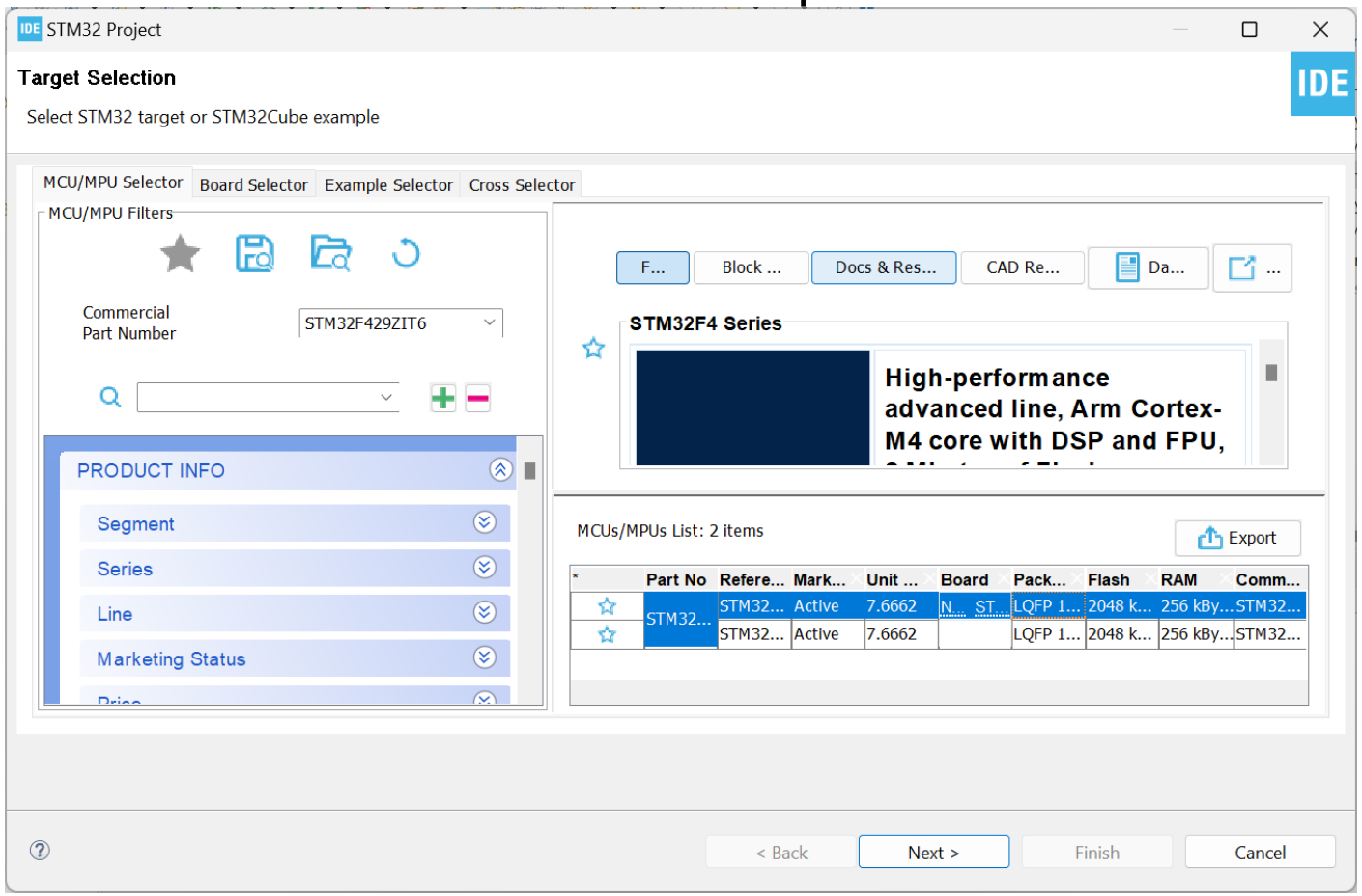
- STM32CubeIDE: IDE + compiler + debugger
- STM32CubeF4: driver + library (install inside CubeIDE)
- ST-LINK009: USB driver (bundled with CubeIDE)

Code example 1: Hello World

- Objective:
  - Create the first project for STM32F4 from scratch
  - Configure GPIO to control an LED with HAL functions

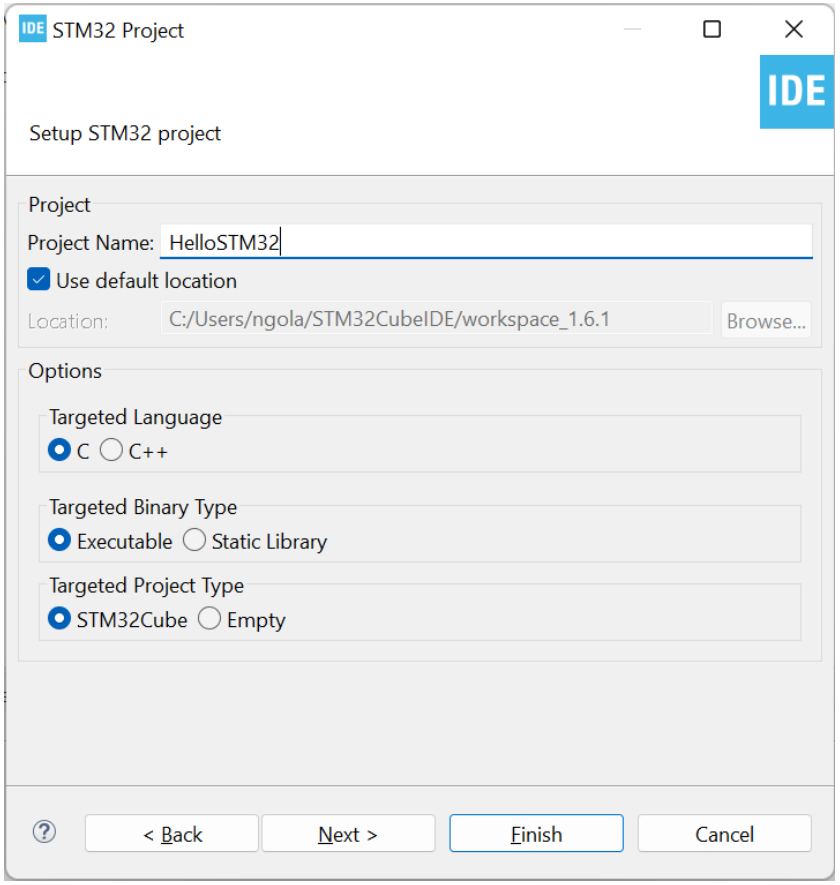
Create a new project

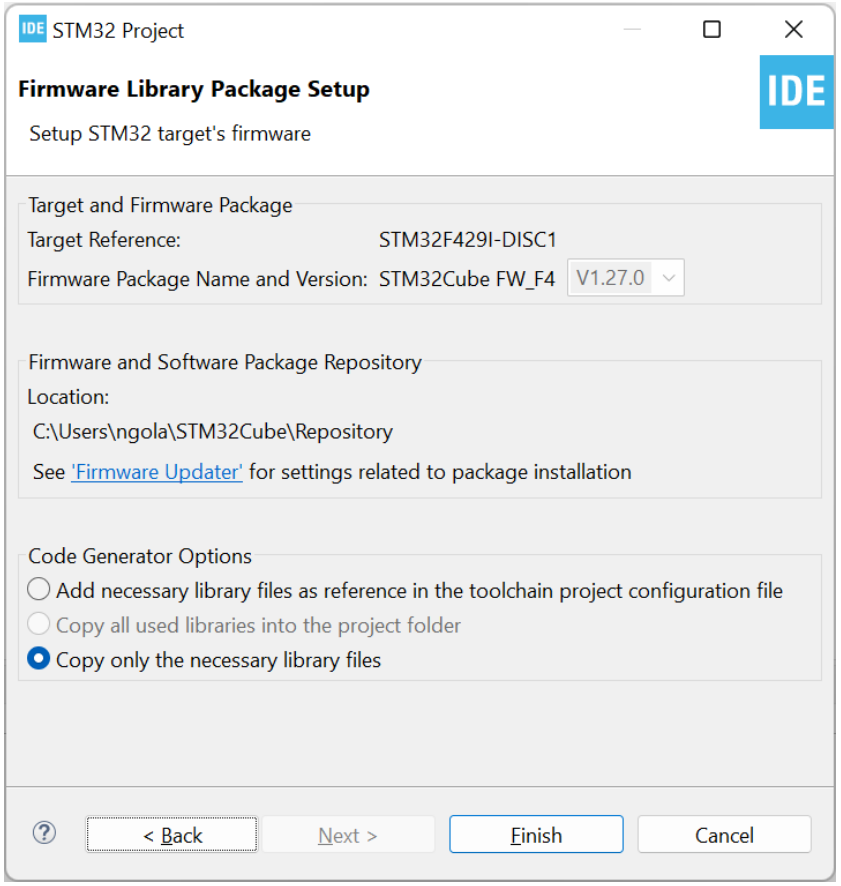
- Menu File → New → STM32 Project → MCU/MPU Selector
  - Search for STM32F429ZIT6 from part number box



Create a new project

- Set project name, firmware package V1.28.0 or V1.28.1
  - Project path must not contain space or special characters



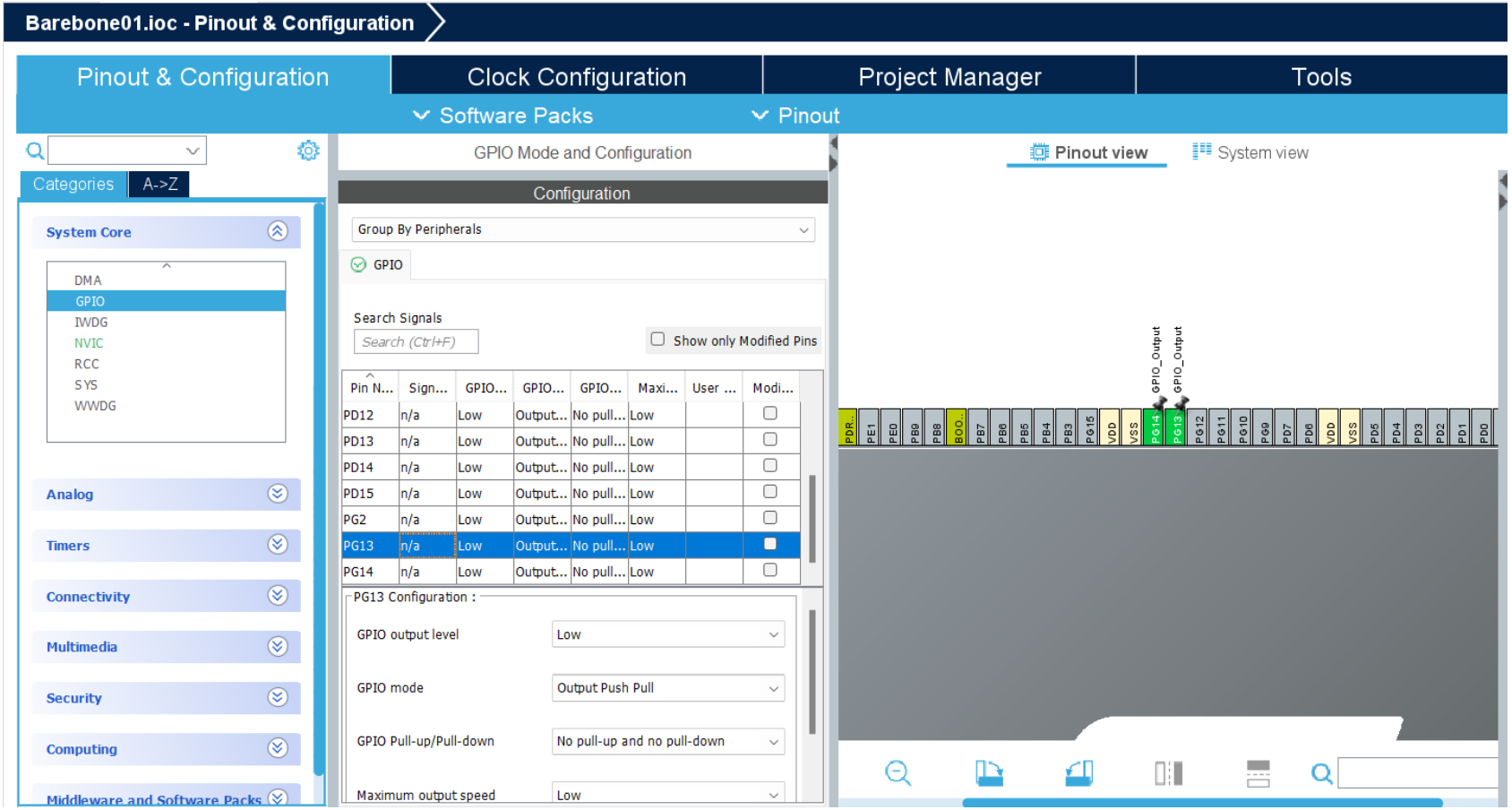


Configure GPIO

- PG13 and PG14 are connected to LED3 and LED4 → need to configure them as output
- Select System Core → GPIO:
  - Set PG13, PG14 to GPIO_Output
- Ctrl+S → save *.ioc file and auto-generate code

Configure GPIO

- PG13 and PG14 as Output, push-pull, Low

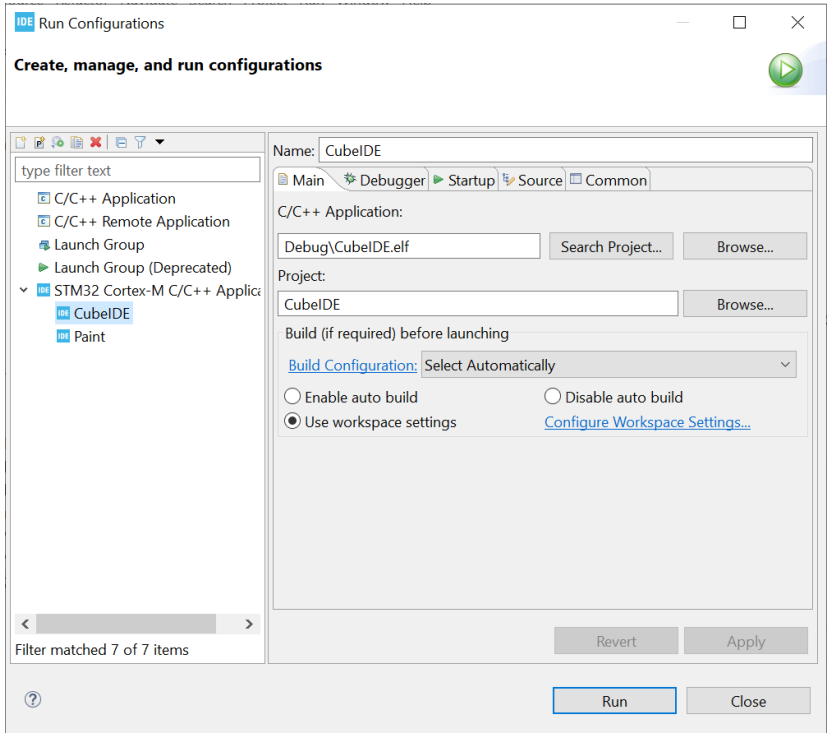


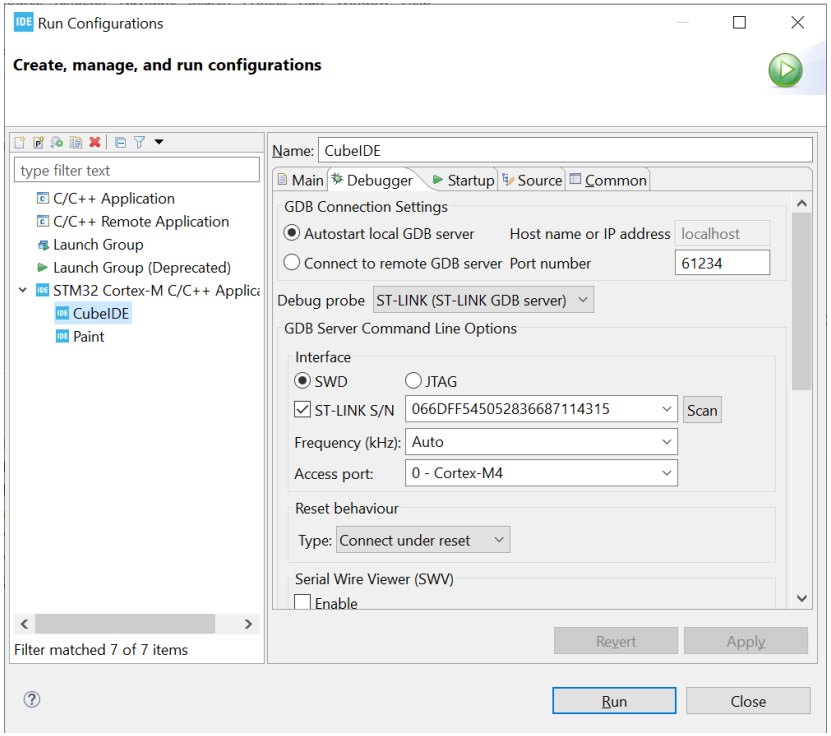
Source code for main()

```
/* Reset of all peripherals, Initializes the Flash
interface and the Systick. */

HAL_Init();

/* Initialize all configured peripherals */

MX_GPIO_Init();

/* Write 1 to GPIO → turn LED ON */

HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13|GPIO_PIN_14,
GPIO_PIN_SET);
```

Build, download, and run

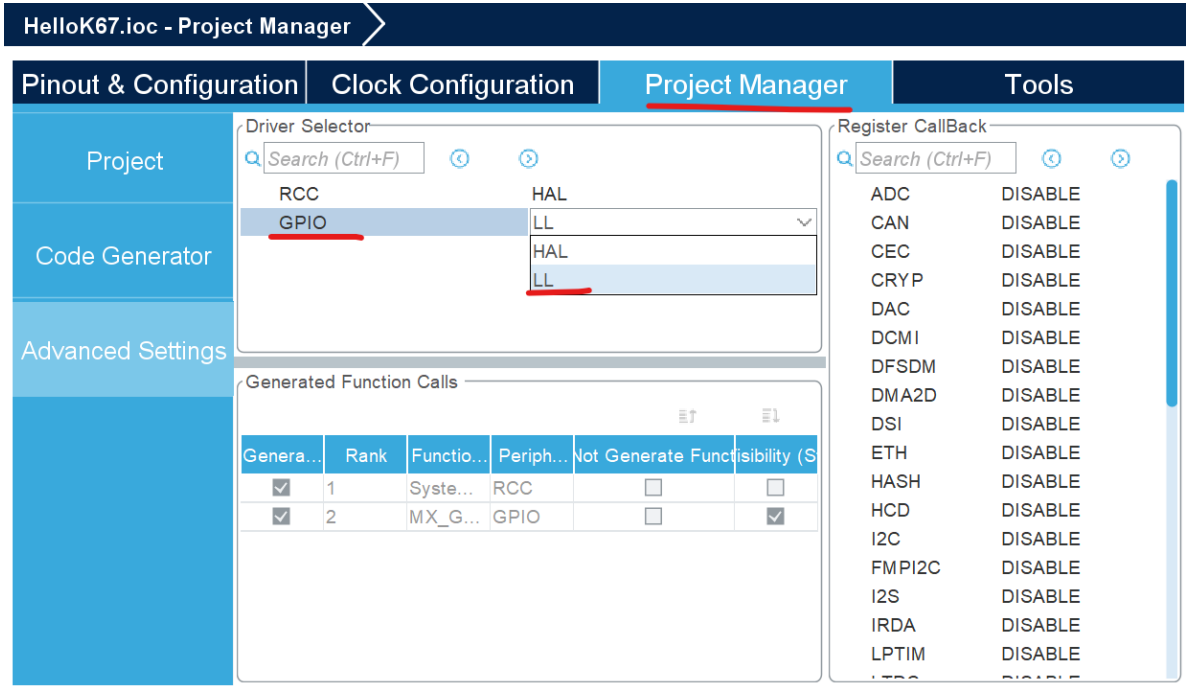
- Run --> Run as STM32 CortexM C/C++ Application
- Plug board to PC's USB port using the mini USB cable.
- Select Debugger as SWD, then Apply and Run
  - If more than one board is connected, select the correct S/N





Analyze the project structure and files

- Code files: *.c, *.h, *.s
- Drivers
- **To-do**: do the same thing (turn on LEDs) but using LL functions instead of HAL functions.



Exercise: LED blinking

- Objective:
  - Develop from previous example code.
  - Turn on/off LEDs periodically

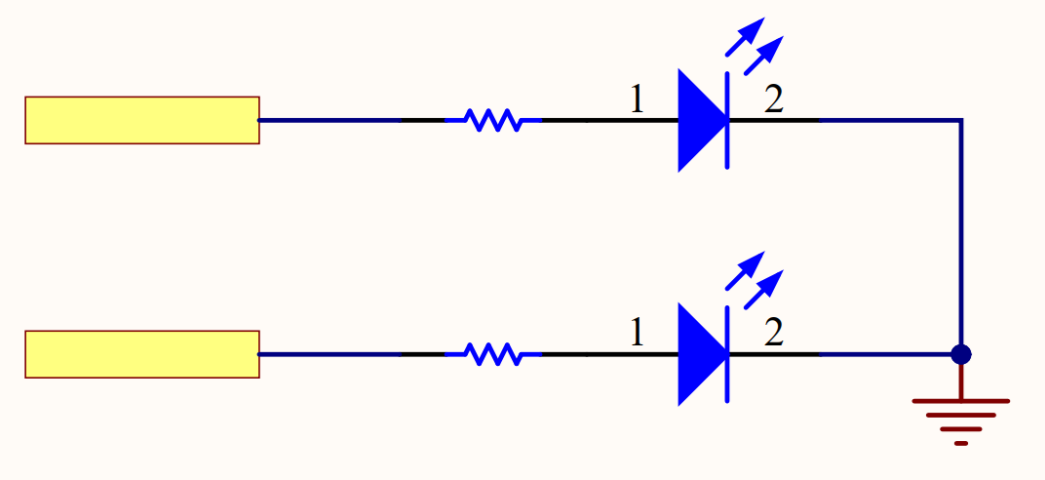
Source code for main loop

```
while (1)
 {
      HAL_Delay(500);
      HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
      HAL_Delay(500);
      HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
 }
```

Note: the above code can be simplified a bit by using HAL_GPIO_TogglePin()

Exercise: Interfacing LED with GPIO

- Assemble the below circuit on breadboard.
  - Select suitable pins on the extension header P1, P2.
  - Connect selected pins with the LED (**don't forget the 330 ohm** resistors).
- Write code to blink the 2 LEDs with different frequencies.
  - Configure GPIOs
  - Write code in main loop.

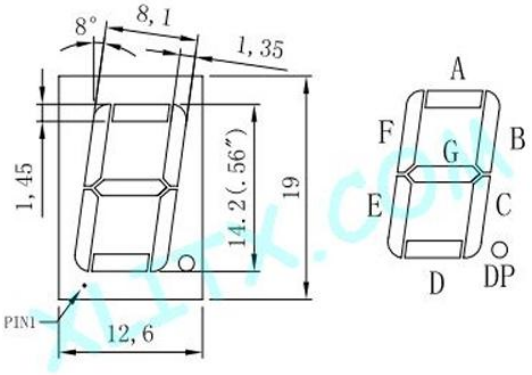


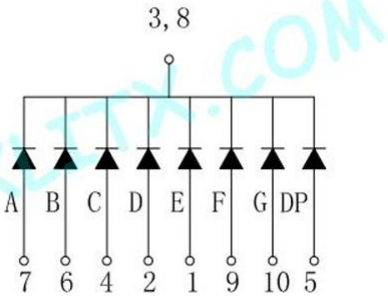
Exercise: Input with GPIO

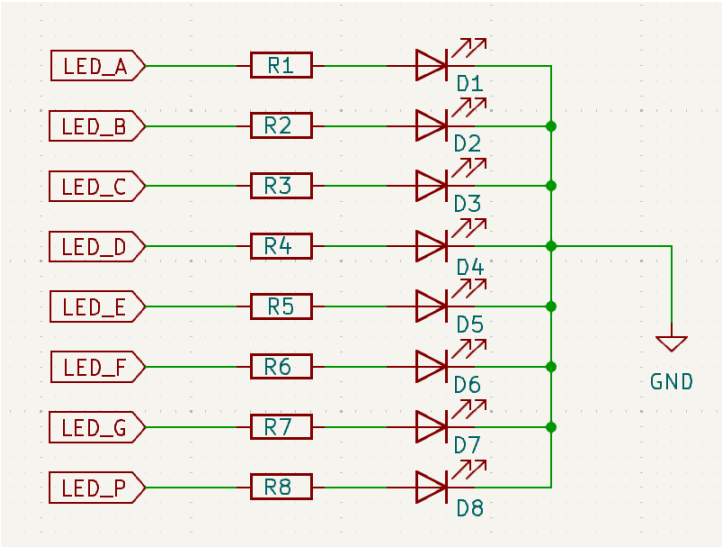
- Update the above project so that:
  - The LEDs blinks only when the Blue button is pressed.
  - The LEDs are permanently ON when the Blue button is released.

Exercise 2: 7 segments LED interfacing

- 5161AS module (common-cathode)

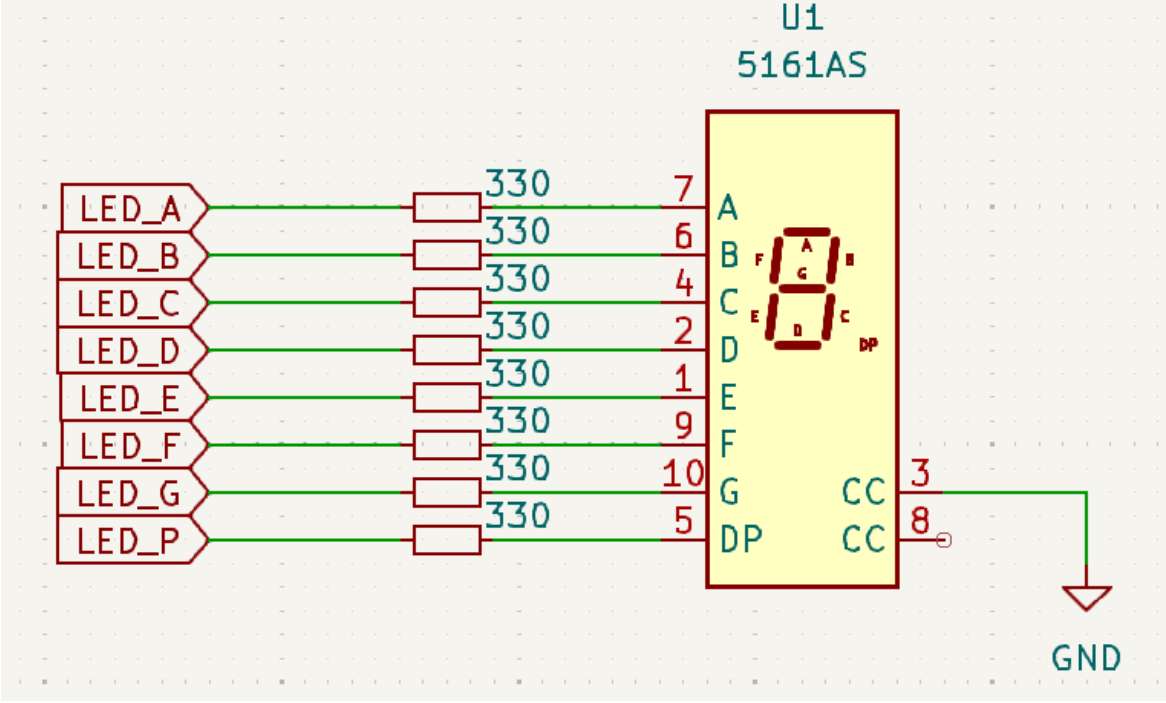






Exercise 2: 7 segments LED interfacing

- Build a circuit for controlling a 7-seg LED module based on the following schematic.
  - GPIO pins for LED_A to LED_P can be selected from the free STM32F429 pins on headers P1 and P2.
- Write a program to display the digits 0 to 9 sequentially.

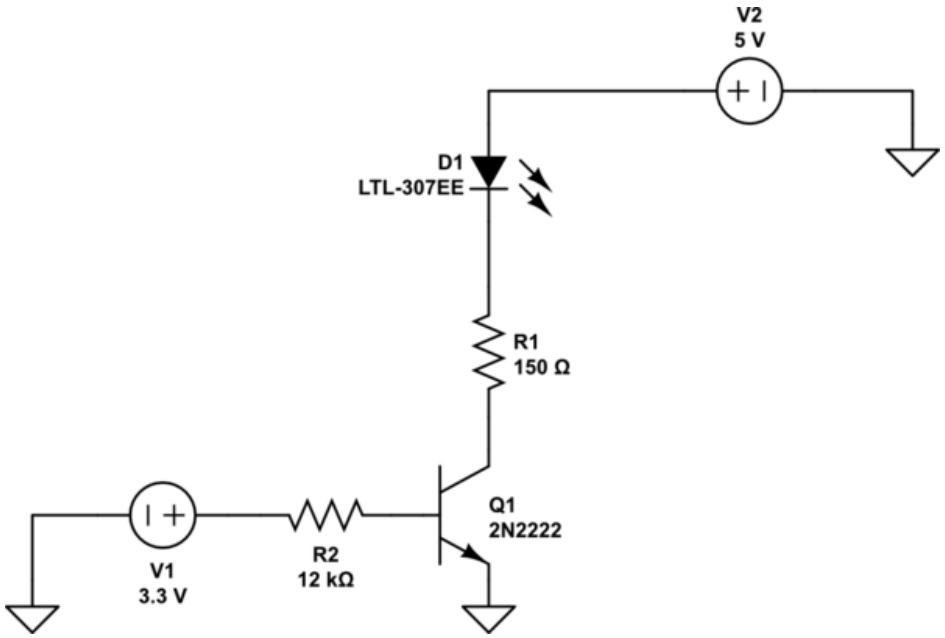


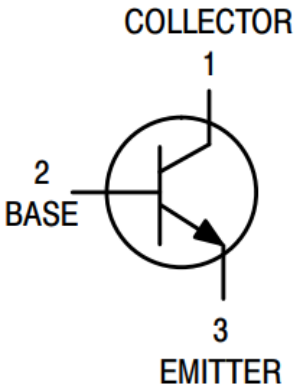
Exercise 3: 7 segments LED array interfacing

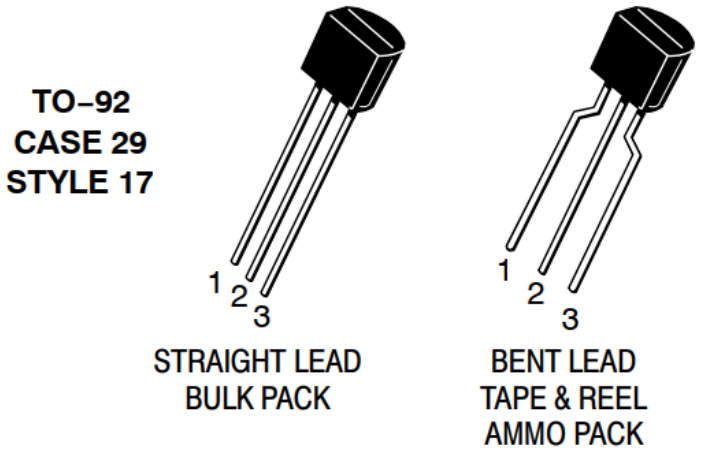
- To display multi-digit numbers: use an array of multiple 7-seg LEDs.
  - Each 7-seg LED requires 8 GPIO pins. How about an array of 10?
  - Need to reduce the number of GPIO pins to control the 7-seg LED array.
- Principal: LED scanning
  - Hardware:
    - All 7-seg modules share the same data bus to control LED segments.
    - Each module have a separate control signal to turn on/off the whole module.
  - Software: LED scanning process
    - Turn on one module at a time for a short time, then sequentially move to the next module in the array.
    - Display the corresponding digit on each module when it is turned on.
    - Repeat the above process indefinitely with a “suitable” frequency and delay time, the “retinal persistence” effect will cause the illusion as if all modules are turned on simultaneously.

Exercise 3: 7 segments LED array interfacing

- Turn on/off LEDs with NPN transistor

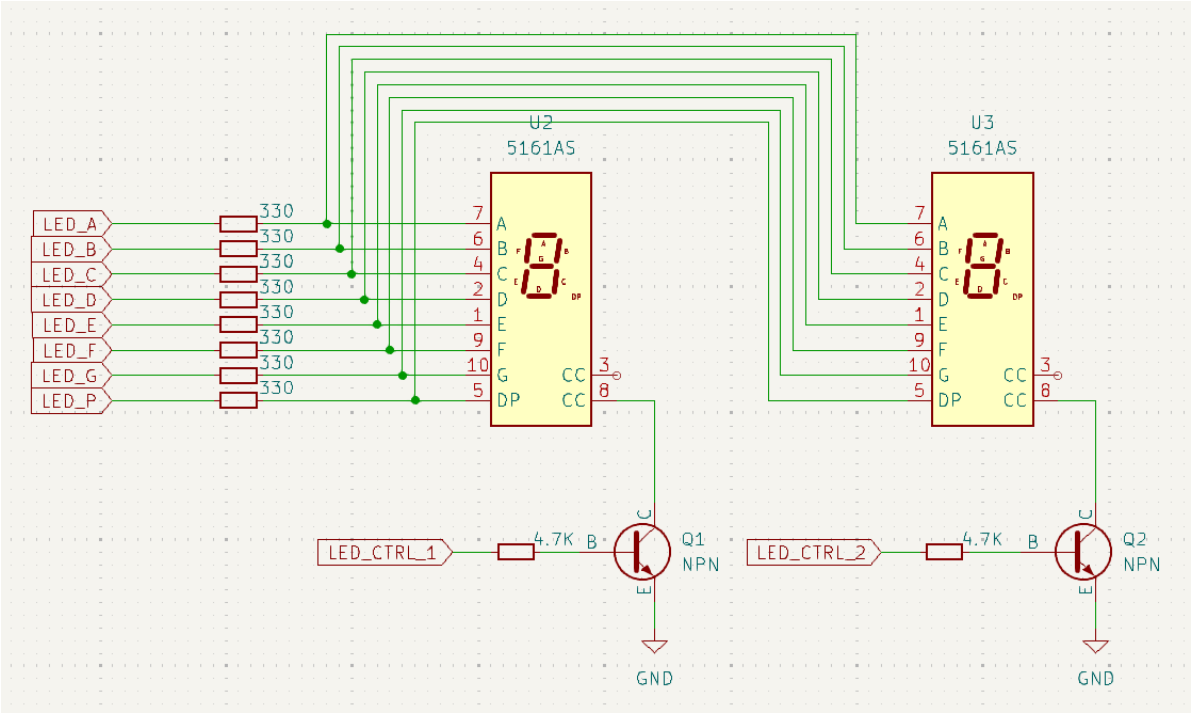






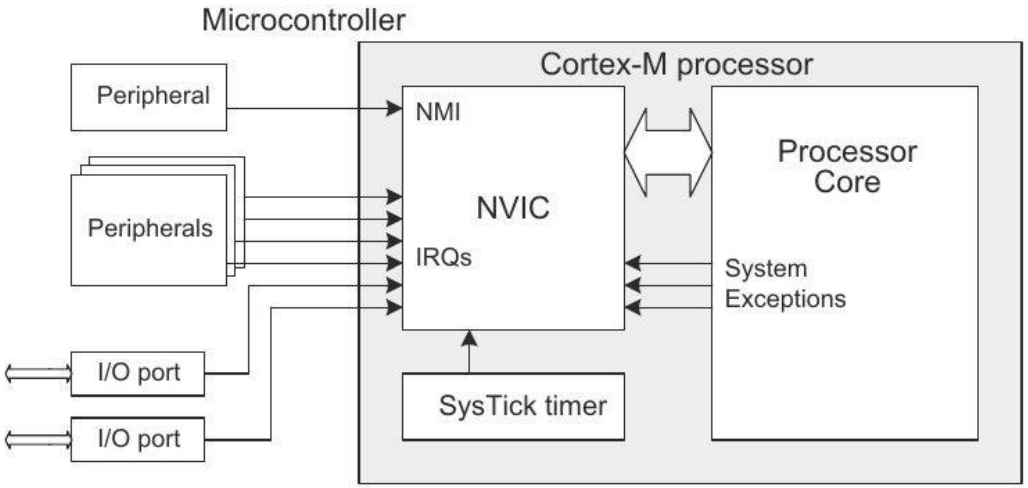
Exercise 3: 7 segments LED array interfacing

- Reference schematic to drive two 7-seg LEDs is below.
- Select the suitable GPIO pins to drive each segment, and to control the two 7-seg modules.
- Configure the selected GPIO pins and write code to display the counting values from 00 to 99 on the two modules.



Interrupt handling

- Interrupt: very important activity, triggers the execution of a specific handler (code) on CPU when a specific event occurs.
- Event: similar to interrupt, but it signals a peripheral to do a hardware function instead of code execution on CPU.
- STM32F4 has a lot of interrupts/events, managed by the NVIC
  - GPIO, peripherals (timers, ADC, DAC,…)
  - SysTick, NMI.
  - 91 programmable interrupts, 16 priority levels



Exercise

- Table 63. Vector table for STM32F42xxx “RM0090 Reference”: list all interrupts that are supported by STM32F429.
- How are they supported by software?

Exercise

- What is the external interrupts EXTIs?
- How many external interrupts can work simultaneously?
- See page 383 “RM0090 Reference” and describe how to set up the configurations for an external interrupt.

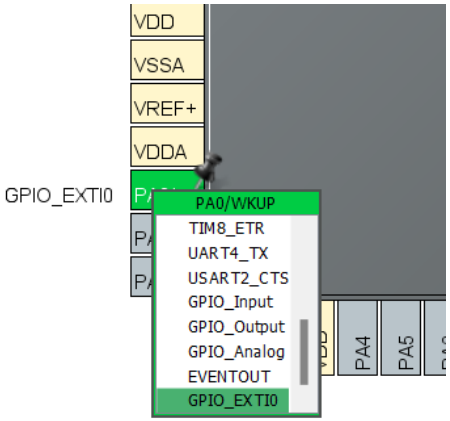
Exercise 4: Handing external interrupt

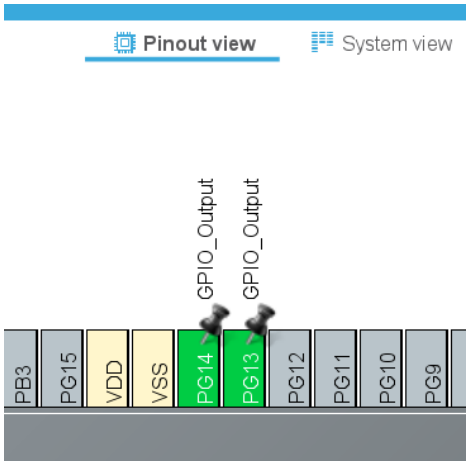
- Set up the external interrupt corresponding to the PA0 pin and handle that interrupt event in software the Green and Red LEDs are toggled when the Blue button is pressed.
- Set up mode for PA0 as GPIO_EXTI0
  - Set working mode: External Interrupt Mode with Rising edge
- Enable the interrupt handler in NVIC
- Write code for the interrupt handler in *_it.c

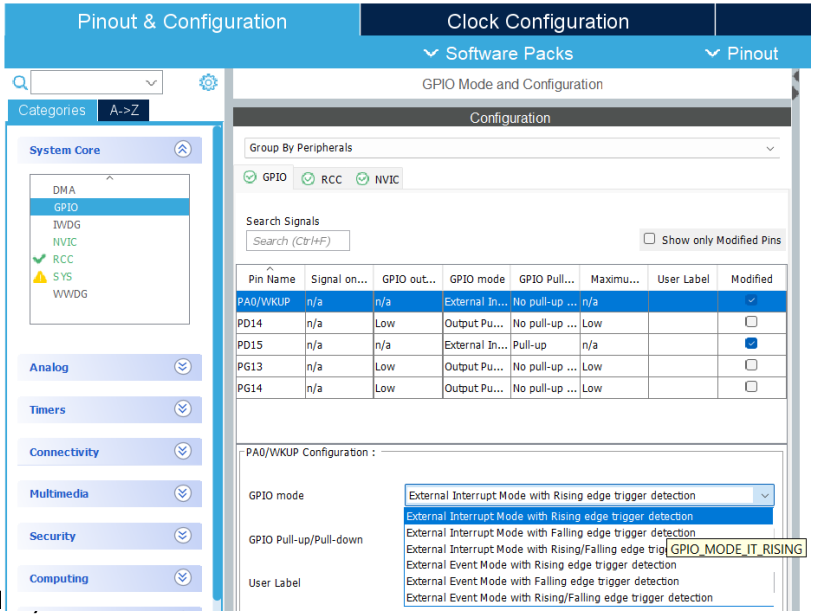
```
void EXTI0_IRQHandler(void)
```

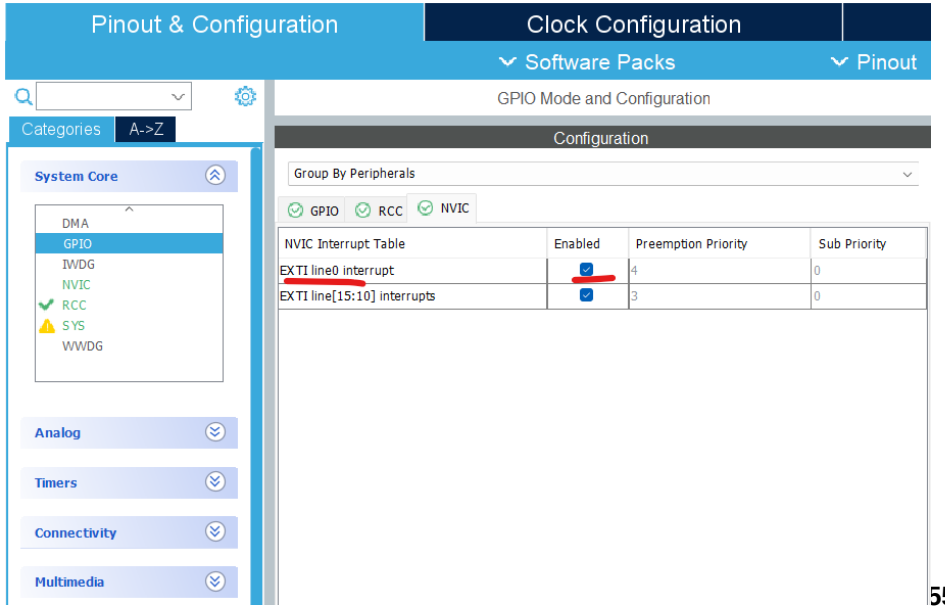
Exercise 4: Handing external interrupt

- Set up GPIO configuration for PA0, PG13, PG14









Exercise 4: Handing external interrupt

- Write code for the EXTI0_IRQHandler() function
- How it works:
  - `MX_GPIO_Init`: set up PA0 with rising edge detection.
  - **`EXTI0_IRQHandler`** : the routine to be called automatically when the EXIT0 interrupt is raised, corresponding to when the button is pressed down.

main.c | *stm32f4xx_it.c | startup_stm32f429zitx.s

```
203   */
204 void EXTI0_IRQHandler(void)
205 {
206   /* USER CODE BEGIN EXTI0_IRQn 0 */
207     HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
208   /* USER CODE END EXTI0_IRQn 0 */
209   HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
210   /* USER CODE BEGIN EXTI0_IRQn 1 */
211
212   /* USER CODE END EXTI0_IRQn 1 */
213 }
```

Working with Timer

- Timer: hardware to do things related to counting time or generating intervals and related functions.
- Application of timer: uncountable!
  - Generating events/interrupts, trigger hardware functions with specific interval/frequency.
    - System timer (systick).
    - ADC, DAC, wake-up low power CPU,...
    - Pulse Width Modulation (PWM).
    - Watchdog.
  - Measuring time intervals.
  - Motor control, encoder interfacing, positioning...
  - Switching for digital power converter…
- STM32F429: 17 timers (p.35-36 Datasheet)
  - Advanced, General purpose, Basic, Watchdogs, Systick.

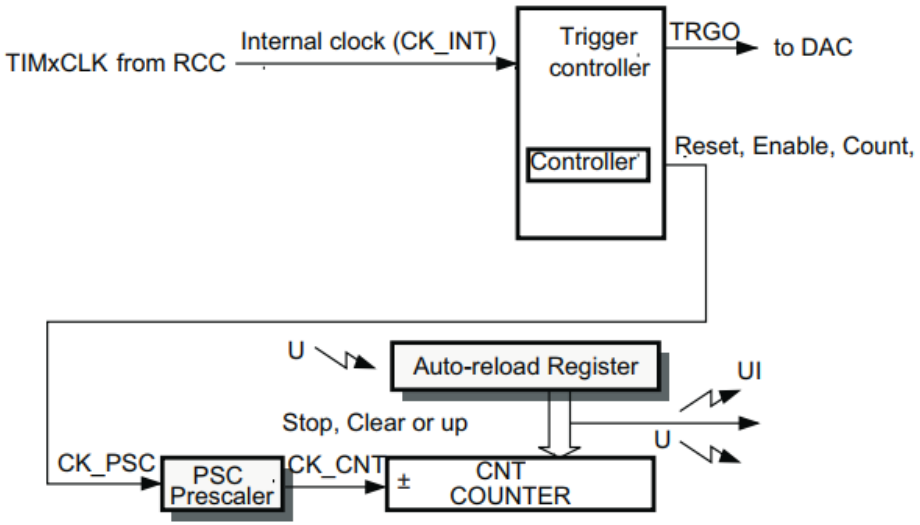
Working with Timer

- General principal:
  - Organization:
    - Clock source: from system clock (bus) or external source.
    - Timer counter registers.
    - Configuration registers.
  - Counter register increase value after each clock period.
  - When the counter is carried out, or equals to some threshold:
    - An event is generated to trigger another peripheral (ADC, DAC…)
    - Or an interrupt is raised to trigger code execution on CPU.
- Programming with Timer: APIs from HAL or LL
  - HAL_TIM (Time Base, Input Capture, Output Compare, PWM...)
  - LL_TIM, LL_LPTIM
  - HAL_WWDG, LL_WWDG

Basic timer: TIM6 and TIM7

- The basic timers with 16-bit auto-reload counter driven by a 16 bit programmable prescaler.
- Able to drive the digital-to-analog converter (DAC).
- Basic operation: clock signal from APB1 divided before generating interrupt/event.
  - Prescaler (16 bit)
  - Counter period (16 bit)

Timer interrupt frequency = f_{APB} / (Prescaler+1) / (Counter Period + 1)



Basic timer: TIM6 and TIM7

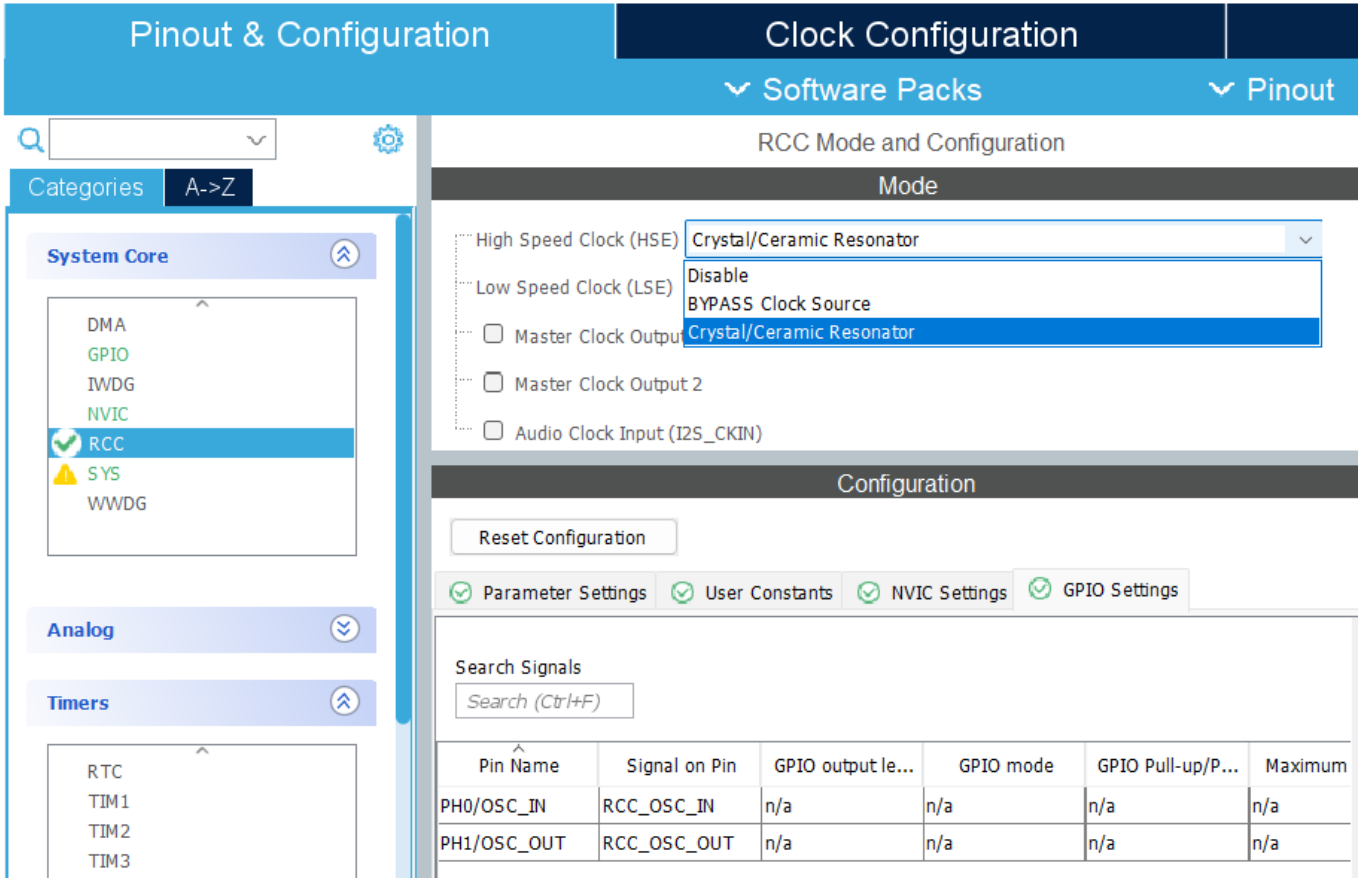
- API: HAL_TIM_Base
  - HAL_TIM_Base_Init()
  - HAL_TIM_Base_DeInit()
  - HAL_TIM_Base_MspInit()
  - HAL_TIM_Base_MspDeInit()
  - HAL_TIM_Base_Start()
  - HAL_TIM_Base_Stop()
  - HAL_TIM_Base_Start_IT()
  - HAL_TIM_Base_Stop_IT()
  - HAL_TIM_Base_Start_DMA()
  - HAL_TIM_Base_Stop_DMA()

Working with basic timers

- Exercise
  - Set up system clock at 180 MHz
  - Write code to blink Red/Green LEDs at 1 Hz

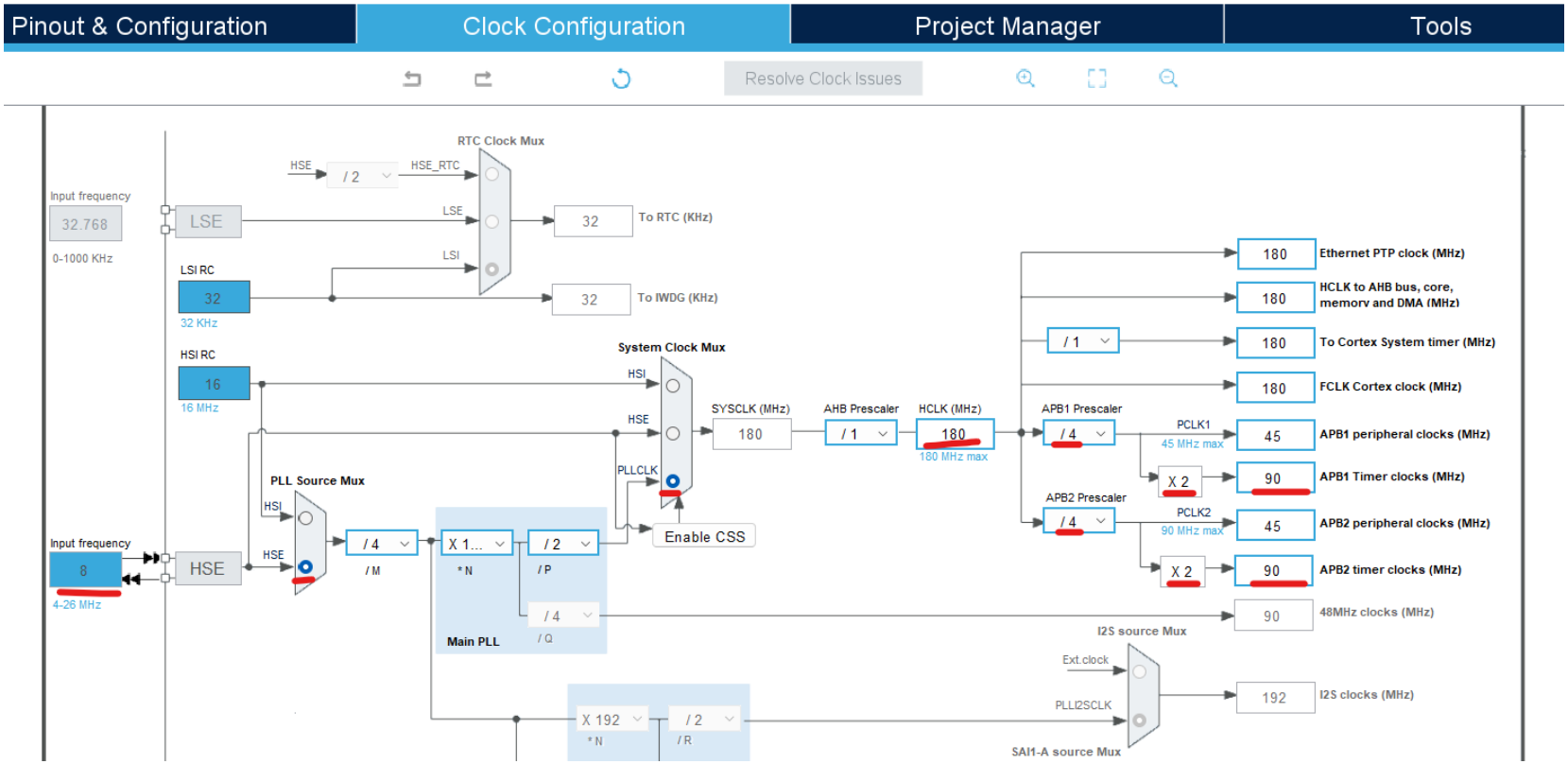
Configuring system clock

- Set up HSE as Crystal/Ceramic Resonator to use the 8 MHz clock from external crystal.



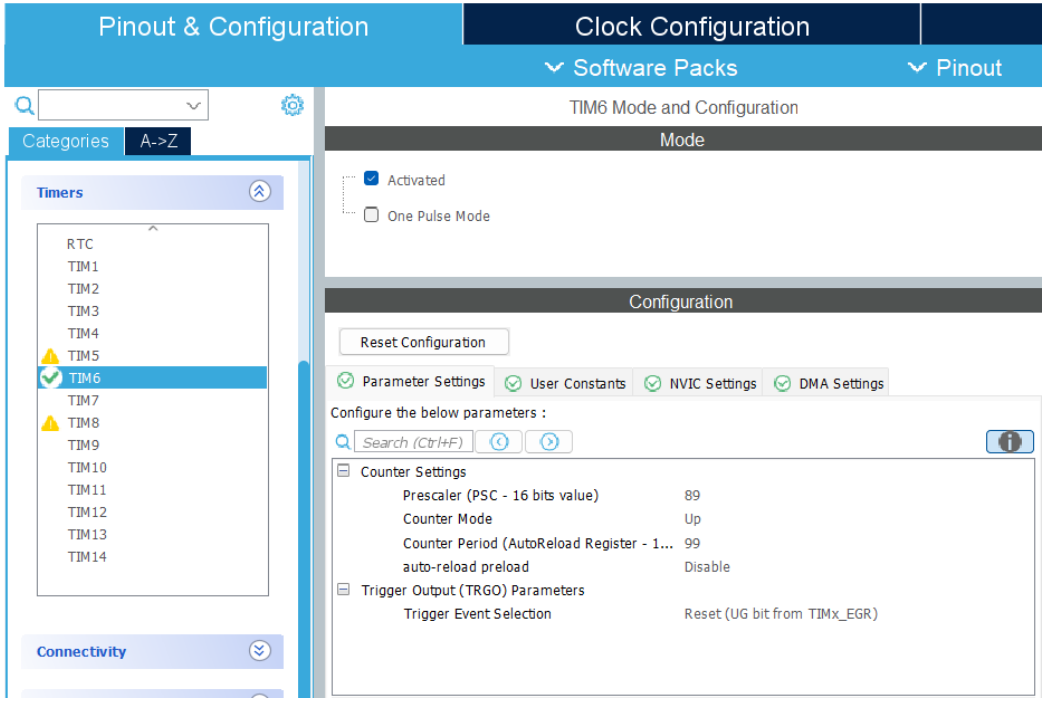
Configuring system clock

- Set up clock rate for CPU and buses
  - HCLK: 180 MHz (for CPU)
  - APB1, APB2 timer clocks: 90 MHz (TIM6 and TIM7 use APB1)



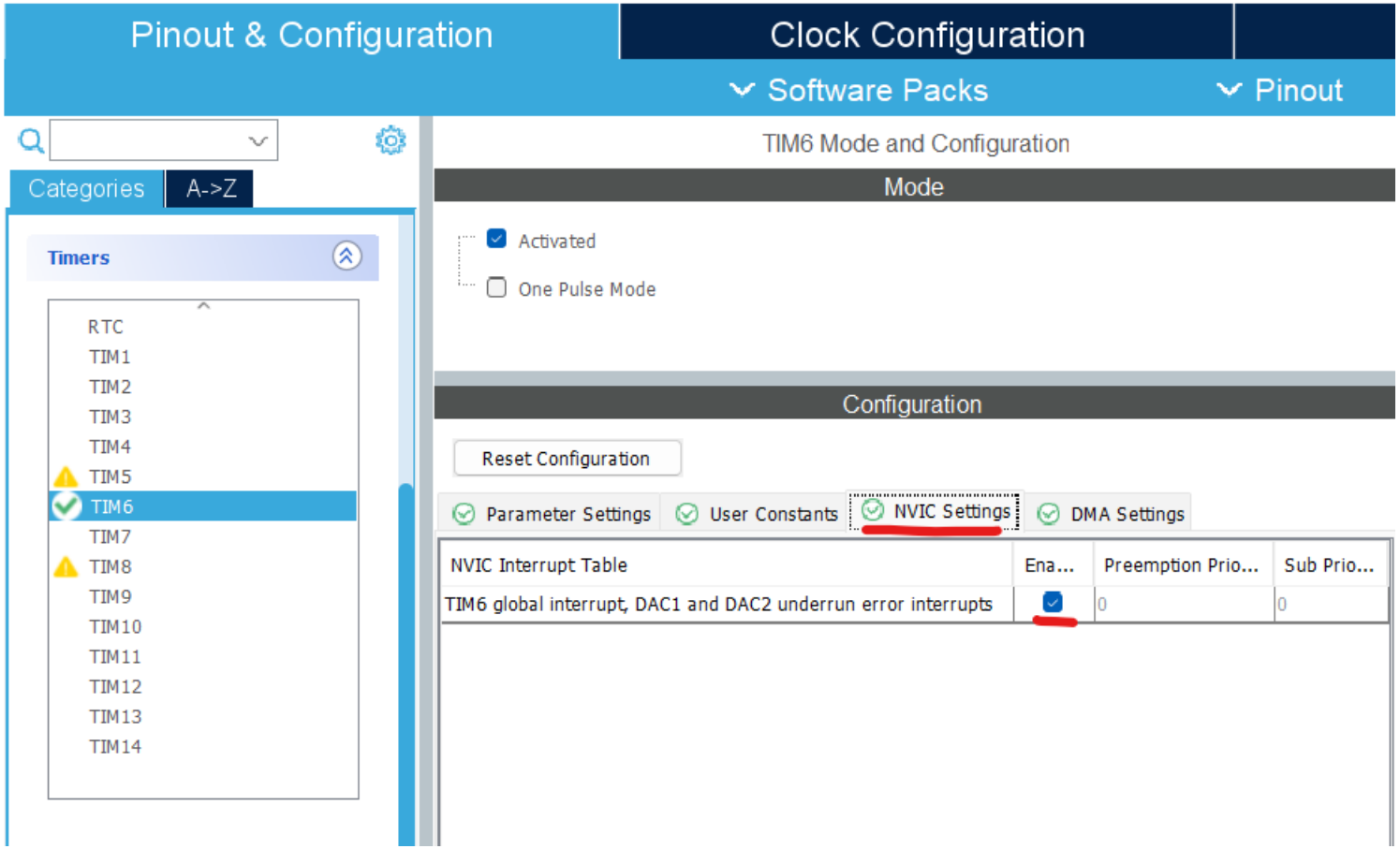
Configuring timer 6: frequency

- Activate TIM6
- Prescaler = 89, Counter Period = 99
  - Timer frequency = 90 MHz / 90 / 100 = 10 KHz
  - What are appropriate values for frequency of 200 Hz?



Configuring timer 6: interrupt hander

- Enable the TIM6 interrupt handler in NVIC settings



Handing the timer 6 interrupt

- Start the timer 6

main.c | stm32f4xx_it.c | startup_stm32f429zitx.s | InterruptEx.ioc

```
87   /* USER CODE END SysInit */
88
89   /* Initialize all configured peripherals */
90   MX_GPIO_Init();
91   MX_TIM6_Init();
92   /* USER CODE BEGIN 2 */
93   HAL_TIM_Base_Start_IT(&htim6);
94   /* USER CODE END 2 */
95
96   /* Infinite loop */
97   /* USER CODE BEGIN WHILE */
```

Handing the timer 6 interrupt

- Global variable: tim6_count in main.c, extern in ***_it.c

main.c | stm32f4xx_it.c | startup_stm32f429zitx.s | InterruptEx.ioc

```
45 /* USER CODE BEGIN PV */
46 int tim6_count;
47 /* USER CODE END PV */
48
```

main.c | stm32f4xx_it.c | startup_stm32f429zitx.s | InterruptEx.ioc

```
56
57 /* External variables ------------
58 extern TIM_HandleTypeDef htim6;
59 /* USER CODE BEGIN EV */
60 extern int tim6_count;
61 /* USER CODE END EV */
62
```

main.c | stm32f4xx_it.c | startup_stm32f429zitx.s | InterruptEx.ioc

```
243 void TIM6_DAC_IRQHandler(void)
244 {
245   /* USER CODE BEGIN TIM6_DAC_IRQn 0 */
246     tim6_count++;
247     if (tim6_count == 5000)
248     {
249         tim6_count = 0;
250         HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
251     }
252   /* USER CODE END TIM6_DAC_IRQn 0 */
253   HAL_TIM_IRQHandler(&htim6);
254   /* USER CODE BEGIN TIM6_DAC_IRQn 1 */
255
256   /* USER CODE END TIM6_DAC_IRQn 1 */
257 }
```

Exercise

- Configure PA0 as GPIO_Input.
- Write code to toggle the red LED when the Blue button (connected to PA0) is double-clicked.
  - Double-click condition: two consecutive button click with less than 300 ms interval.
- Reference sample code:

main.c | stm32f4xx_it.c | startup_stm32f429zitx.s | InterruptEx.ioc

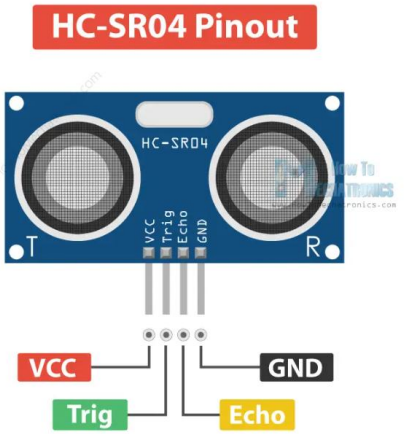
```
204 void EXTI0_IRQHandler(void)
205 {
206   /* USER CODE BEGIN EXTI0_IRQn 0 */
207     if (tim6_count < 3000)
208     {
209         HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
210     }
211     tim6_count = 0;
212
213   /* USER CODE END EXTI0_IRQn 0 */
214   HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
215   /* USER CODE BEGIN EXTI0_IRQn 1 */
216
217   /* USER CODE END EXTI0_IRQn 1 */
218 }
```

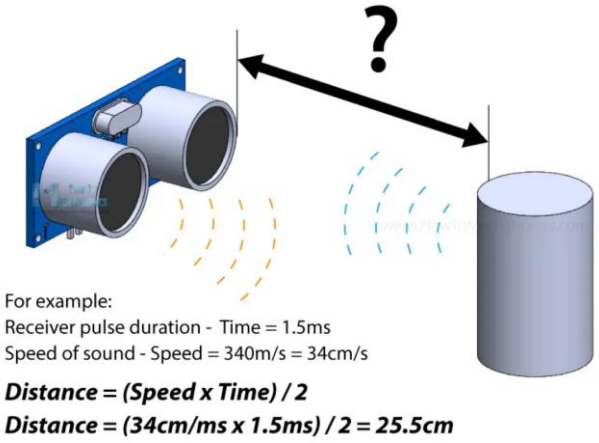
Exercise: interfacing with ultrasonic range sensor

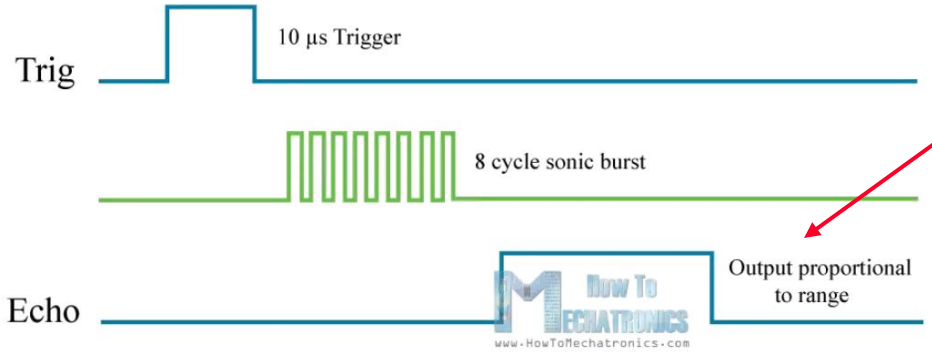
- Connect STM32F429 with the HC-SR04 ultrasonic sensor.
- Write code to measure the distance from the sensor to the obstacle when the Blue button is pressed.
- This is very easy on Arduino with the ultrasonic sensor library!
- But you need to write your own code in this example, to understand how the sensor works

Exercise: interfacing with ultrasonic range sensor

- Ultrasonic range sensor principal







Cần đo độ rộng xung này

Building the circuit

- Connections
  - HC-SR04 VCC: 5V
  - HC-SR04 GND: GND
  - HC-SR04 Trig: STM32F429 GPIO D14
  - HC-SR04 Echo: STM32F429 GPIO D15
- GPIO configuration on STM32F429
  - GPIO A0: GPIO_EXTI0, to catch user button pressed interrupt.
  - D14: GPIO_Output, to generate the Trig pulse.
  - D15: GPIO_EXTI15, Rising edge and Falling edge to catch the interrupt on both sides of the Echo pulse.
  - Enable the D15 EXTI line 15 interrupt in NVIC

    ➔ need to write code in this interrupt hander to measure the duration of the Echo pulse.

Exercise: interfacing with ultrasonic range sensor

- Generate Trig inside EXTI0 interrupt handler
  - Raise Trig to 1 when Blue button pressed (interrupt occurs).
  - Wait at least 10 us before lowering Trig to 0.
  - Note: do not use HAL_Delay inside an interrupt hander, as HAL_Delay uses timer which can has lower priority than EXTI0.

main.c | stm32f4xx_it.c | startup_stm32f429zitx.s | InterruptEx.ioc | system_stm32f4xx.c

```
204 void EXTI0_IRQHandler(void)
205 {
206   /* USER CODE BEGIN EXTI0_IRQn 0 */
207     int t6count = 100;
208     HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_SET);
209     while (t6count--);  //wait at least 10 us
210     HAL_GPIO_WritePin(GPIOD, GPIO_PIN_14, GPIO_PIN_RESET);
211
212   /* USER CODE END EXTI0_IRQn 0 */
213   HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
214   /* USER CODE BEGIN EXTI0_IRQn 1 */
215
216   /* USER CODE END EXTI0_IRQn 1 */
217 }
```

Exercise: interfacing with ultrasonic range sensor

- Measure the Echo pulse width: EXTI15_10 handler
  - Note: the handler is called on both sides of the pulse.
  - Reset timer 6 value upon rising edge of Echo pulse.
  - Measure the pulse width by timer 6 value upon falling edge.
- Finish the code below to calculate distance

main.c | *stm32f4xx_it.c | startup_stm32f429zitx.s | InterruptEx.ioc | system_stm32f4xx.c

```
229 void EXTI15_10_IRQHandler(void)
230 {
231   /* USER CODE BEGIN EXTI15_10_IRQn 0 */
232     int state = HAL_GPIO_ReadPin(GPIOD, GPIO_PIN_15);
233     if (state == GPIO_PIN_SET)
234         tim6_count = 0;
235     else
236     {
237         int echo = tim6_count;
238     }
239     HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
240   /* USER CODE END EXTI15_10_IRQn 0 */
241   HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_15);
```

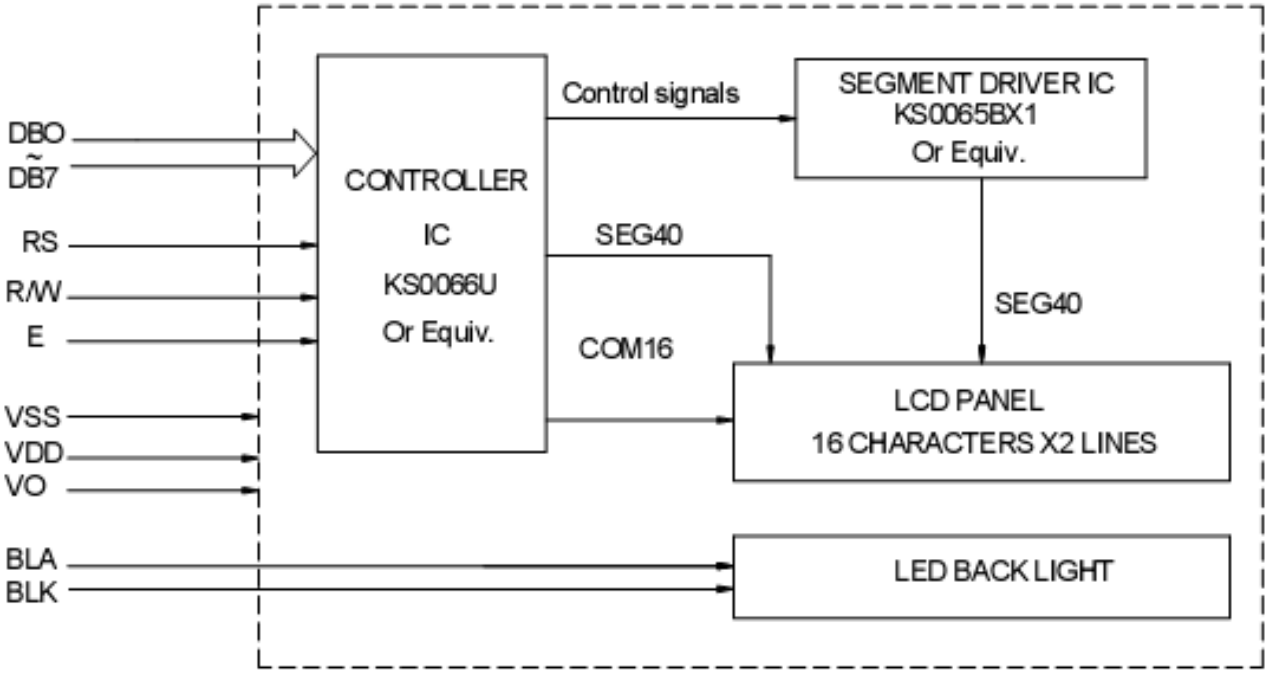
Exercise: interfacing with ultrasonic range sensor

- Develop new function based on the previous exercise:
- Continuously measure X as the distance from sensor to the obstacle:
  - If X < 20 cm: turn on the Red LED, otherwise the Red LED is off.
  - If X > 50 cm: turn on the Green LED, otherwise Green is off.
  - If 20 < X < 50 cm: the Green LED is dimmed, with the light intensity inverse proportional with value of X.
    - X = 50 cm: Green LED turns on completely.
    - X = 20 cm: Green LED turns off completely.

Controlling LCD 1602

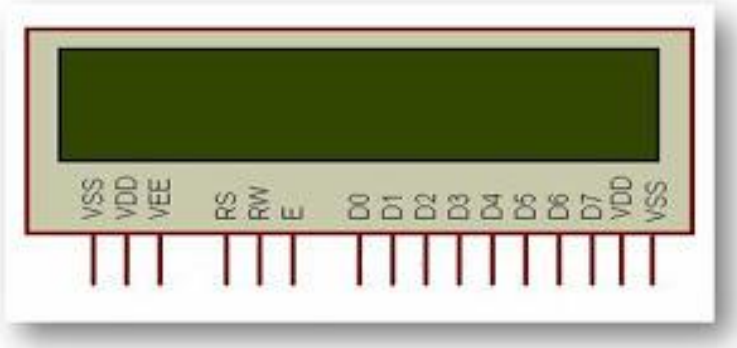
- LCD 1602:
  - Very popular text LCD
  - 2 rows x 16 columns

- Block diagram:



LCD 16x2 pinout

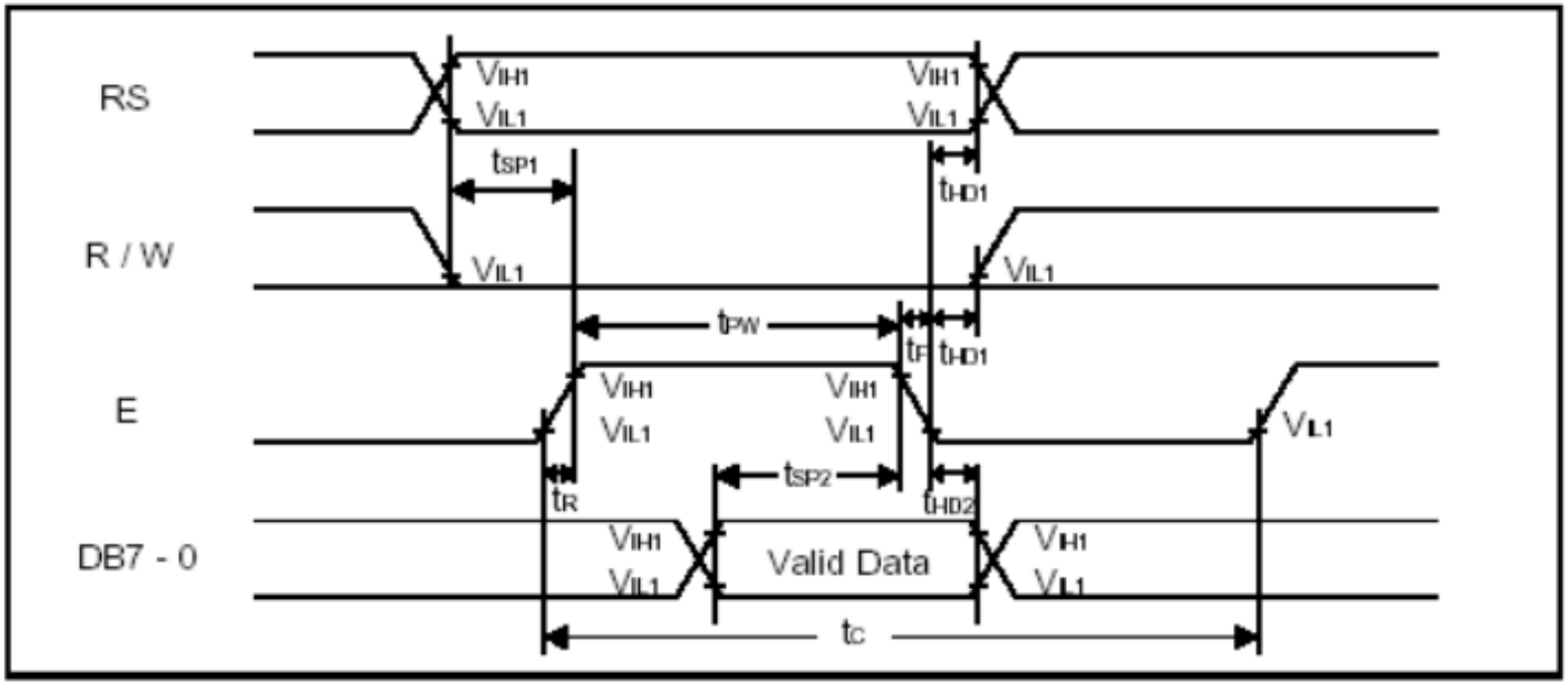
Pin Number	Symbol	Pin Function
1	VSS	Ground
2	VCC	+5v
3	VEE	Contrast adjustment (VO)
4	RS	Register Select. 0:Command, 1: Data
5	R/W	Read/Write R/W=0: Write R/W=1: Read
6	EN	Enable. Falling edge triggered
7	D0	Data Bit 0
8	D1	Data Bit 1
9	D2	Data Bit 2
10	D3	Data Bit 3
11	D4	Data Bit 4
12	D5	Data Bit 5
13	D6	Data Bit 6
14	D7	Data Bit 7/Busy Flag
15	A/LED+	Back-light Anode(+)
16	K/LED-	Back-Light Cathode(-)



Note: need to control contrast

Timing diagram

- Writing
  - RS = 1: data (character code)
  - RS = 0: command



Interfacing with LCD 16x2

- Step 1: send command to LCD
  - R/W = 0 (write)
  - RS = 0 (command)
- Step 2: Send character code to LCD
  - R/W = 0 (write)
  - RS = 1 (data)

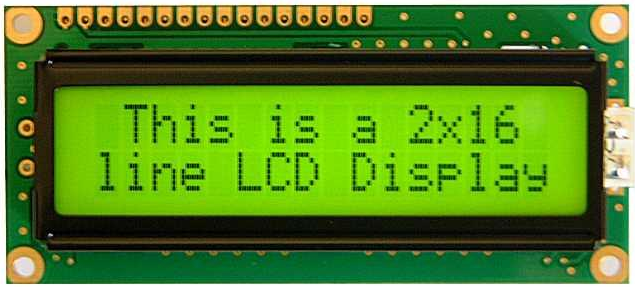
Sample code: send command code

```
void LCD_Send_Command(unsigned char x)
{
  LCD_DATA=x;          //Ma lenh
  RS=0;                //Chon thanh ghi lenh
  RW=0;                //De ghi du lieu
  EN=1;
  Delay_ms(1);
  EN=0;
  Delay_ms(1);
  EN=1;
}
```

Sample code: send character code

```
void LCD_Write_One_Char(unsigned char c)
{
  LCD_DATA=c;          //Gia tri du lieu
  RS=1;                //Chon thanh ghi du lieu
  RW=0;                //Ghi du lieu
  EN=1;
  Delay_ms(1);
  EN=0;
  Delay_ms(1);
  EN=1;
}
```

Command set of LCD 16x2



LCD Command Codes

Code (Hex)	Command to LCD Instruction Register
1	Clear display screen
2	Return home
4	Decrement cursor (shift cursor to left)
6	Increment cursor (shift cursor to right)
5	Shift display right
7	Shift display left
8	Display off, cursor off
A	Display off, cursor on
C	Display on, cursor off
E	Display on, cursor blinking
F	Display on, cursor blinking
10	Shift cursor position to left
14	Shift cursor position to right
18	Shift the entire display to the left
1C	Shift the entire display to the right
80	Force cursor to beginning to 1st line
C0	Force cursor to beginning to 2nd line
38	2 lines and 5x7 matrix

LCD initialization routine

```
void LCD_init()
{
      //Chon che do 5x7 pixel, 2 lines
      LCD_Send_Command(0x38);
      //Bat hien thi, nhap nhay con tro
      LCD_Send_Command(0x0E);
      LCD_Send_Command(0x01); //Xoa man hinh
      LCD_Send_Command(0x80); //Ve dau dong
}
```

Exercise

- Build circuit with pinout as in the table
- Comprehend the sample code
- Write code to display the 2 lines on the LCD

```
Hello K67 ITxxxx

****************
```

- Add the code to shift the display content to left/right

STM32F4	LCD 1602
D0	GPIO D14
D1	GPIO D15
D2	GPIO D12
D3	GPIO D13
D4	GPIO D10
D5	GPIO D11
D6	GPIO D8
D7	GPIO D9
RW	GPIO G2
RS	GPIO G3
E	GPIO G4
VDD	5V
VSS	GND
VEE/VO	GND
A	5V
K	GND

Exercise

- Read distance from ultra-sonic range sensor and display on LCD 1602.

exercise

- Write code to display time on LCD 1602 with the following format:

[hh]:[mm]:[ss]:[s10]

s10: tenth of second

- Note:
  - Time is measure by using timer 6.